Documentation

Impianti di Elaborazione

Antonio Sirignano[*,a,1]

[a] *Università degli studi Federico II*

This manuscript was compile on 5 novembre 2026

Sommario

Progetto per l'esame di Impianti di Elaborazione.
GitHub Link: https://github.com/antsiri/Progetto-di-Impianti-di-Elaborazione-

Keywords: *Capacity Test*

INDICE

1 Introduzione **2**

2 Capacity Test **2**

2.1 Setup esperimento . . . 2
2.2 Descrizione dei test . . . 2
2.3 Grafici e analisi . . . 2
2.4 Analisi della Fairness e della Ripetibilità del Benchmark . . . 3

3 PCA e Clustering **3**

3.1 Obiettivo e Setup . . . 3
3.2 Raccolta dati . . . 3
Generazione dei file campione • Generazione del traffico e misurazione
3.3 Pulizia e caratterizzazione HL/LL (reale) . . . 4
Pulizia del dataset • Analisi delle componenti principali (PCA) • Clustering gerarchico e selezione dei parametri
3.4 Clustering e scalabilità . . . 5
Il problema della scalabilità • Pipeline incrementale a blocchi (*chunking*) • Risultati • Caratterizzazione del workload sintetico tramite tabella di contingenza
3.5 Costruzione del workload sintetico . . . 6
Pesatura delle risorse • Generazione del traffico sintetico
3.6 Caratterizzazione LL (sintetico) . . . 6
Pulizia • PCA e Clustering
3.7 Validazione Statistica . . . 7
Estrazione dei campioni per cluster • Test di normalità • Test di Wilcoxon
3.8 Conclusioni . . . 7

4 Workload Characterization **7**

4.1 Obiettivo e Setup . . . 7
4.2 Ingestione Dati e Pre-processing . . . 8
4.3 Caratterizzazione descrittiva del Carico Reale . . . 8
4.4 Modellazione Statistica Avanzata . . . 9
Analisi delle Correlazioni Inter-Risorsa • Autocorrelazione Temporale • Fitting delle distribuzioni temporali e Test Goodness-of-Fit
4.5 Architettura del generatore di Carico Sintetico . . . 9
Progettazione del Meccanismo IPC • Sincronizzazione e Campionamento Probabilitstico
4.6 Validazione sperimentale e Discussione Critica . . . 10
Confronto Reale vs Sintetico • Analisi Quantitativa delle Medie ed Errore Relativo • Limiti e Lavori futuri
4.7 Conclusioni . . . 10

5 DoE **11**

5.1 Introduzione . . . 11
5.2 Progettazione dell'Esperimento (Design of Experiments) . . . 11
identificazione di Fattori, Livelli e Risposta • Pianificazione Sperimentale (Full Factorial Design) • Architettura di Test in Python
5.3 Analisi del Dataset e Modello di Regressione Lineare . . . 11
Sintesi del Modello OLS • Analisi della Varianza (Tabella ANOVA) • Analisi di Importanza dei Fattori
5.4 Analisi di Significativitá e Verifica delle Ipotesi . . . 12
Test di Normalità sui Residui (Shapiro-Wilk) • Test Non Parametrico di Kruskal-Wallis
5.5 Conclusioni . . . 12

6 Regression **12**

6.1 Introduzione . . . 12
6.2 Strumenti . . . 12
6.3 Metodologia . . . 12
Stima lineare • Controllo della normalità dei residui • Test di trend: τ di Kendall (Mann-Kendall) • Stimatore di Thell-Sen • Failure prediction
6.4 Preparazione dei dati . . . 13
6.5 Risultati . . . 13
EXP2 • os1 • os2 • os3 • Confronto tra os1, os2 e os3 • VMres1, VMres2, VMres3
6.6 Failure Prediction . . . 17
6.7 Validazione . . . 17
6.8 Conclusioni . . . 17

7 Benchmark **17**

7.1 Introduzione . . . 17
7.2 Analisi Descrittiva dei Dati Campionari . . . 18
7.3 Valutazione della Dimensione Campionaria Teorica (n) . . . 18
7.4 Pipeline di Inferenza Statistica . . . 18
Test di Normalità (Shapiro-Wilk) • Test di Omoschedasticità (Brown-Forsythe / Levene) • Confronto tra le prestazioni

8 FFDA **19**

8.1 Introduzione . . . 19
8.2 Metodologia . . . 19
8.3 Componenti . . . 19
8.4 Risultati . . . 20

9 RBD & Fault Tree **20**

9.1 Esercizio 1 . . . 20
9.2 Esercizio 2 . . . 21
9.3 Esercizio 3 . . . 21
9.4 Esercizio 4 . . . 22
9.5 Esercizio 5 . . . 22

[*] Corresponding author: ant.sirignano@studenti.unina.it (A. Sirignano).
[†] These authors contributed equally to this work.

This document class was prepared on VSCode and compiled with pdfLaTeX. No errors were found during the compilation process. Rho-class supports external editors, though additional setup may be required.

1. INTRODUZIONE

In questa documentazione vengono spiegati e risolti problemi legati agli Impianti di Elaborazione.

2. CAPACITY TEST

2.1. Setup esperimento

Lo scopo del seguente esperimento p quello di utilizzare la **massima capacità** di un Web Server.
A tale scopo, è stata impostata una piattaforma sperimentale su **macchina virtuale** che ospita un sistema operativo **Linux Ubuntu 20.04**, il quale a sua volta ha in esecuzione un servizio **Apache2** che funge da gestore del web server.
Per simulare i processi di richieste di un determianto **carico** sul web server, vengono generate dei file di varia dimesione utilizzando il comando `allocate`. Le dimesioni variano da pochi kiloByte fino a cento MegaByte.
La **caratterizzazione della capacity** è un test che richiede un certo numero di queste risorse per valutare la stabilità del sistema a fronte di un crescente tasso di richieste.
Le specifiche tecniche della macchina virtuale su cui è ospitato il web server sono le seguenti:

Tabella 1. Caratteristiche della Macchina Virtuale (UTM)

Categoria	Parametro	Valore
Sistema	Architettura	x86_64
Sistema	Macchina Emulata	Standard PC (Q35 + ICH9, 2009)
Sistema	Sistema Operativo	Ubuntu 22.04.5 LTS
Hardware	Memoria RAM	10 GB
Hardware	Dimensione Disco	25,75 GB
Rete	Tipologia Rete	Shared Network (e1000)

Note: Dati rilevati dal gestore di macchine virtuali UTM su host Mac.

Per l'esecuzione dei test, è stato utilizzato **Apache JMeter 21.0.12**, eseguito sulla macchina Host con le seguenti specifiche:

Tabella 2. Caratteristiche della Macchina Host (MacBook Pro)

Categoria	Parametro	Valore
Hardware	Modello	MacBook Pro (16-inch, 2019)
Hardware	Processore	2,3 GHz 8-Core Intel Core i9
Hardware	Memoria RAM	16 GB 2667 MHz DDR4
Hardware	Scheda Grafica	AMD Radeon Pro 5500M 4 GB
Hardware	Archiviazione	1 TB (Macintosh HD)
Sistema	Sistema Operativo	macOS Tahoe (v26.6)

Note: Dati rilevati dalle impostazioni di sistema della macchina Host.

2.2. Descrizione dei test

Il test si compone di un singolo **Thread Group**. Il gruppo ha la seguente configurazione:

- `Number of Threads (users)`: 50
- `Ramp-up Period (seconds)`: 25
- `Duration (seconds)`: 300

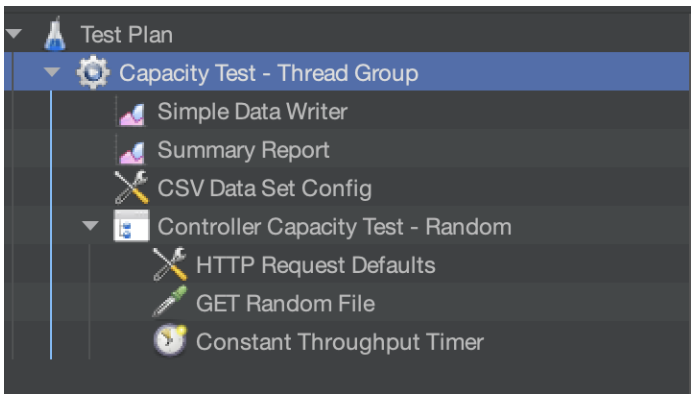
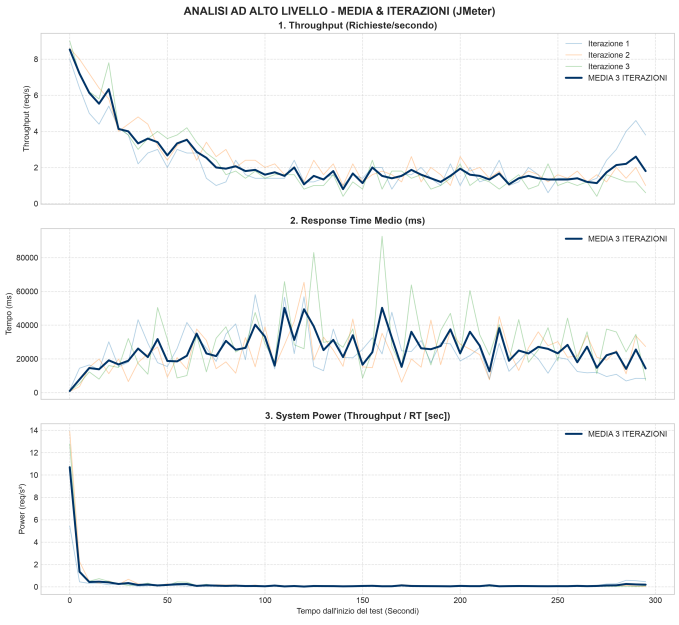


Figura 1. JMeter Test Plan.

Nel test è stato utilizzato uno script che genera file di diverse dimesione sul server. Al fine di tenere traccia dei file generati, viene scritto un file `.csv` che contiene il path di ogni singolo file. Per questo nel test vi è un **CSV Data Set Config**, il quale legge ogni riga corrispondente al filepath di ogni file generato.
All'interno del gruppo è presente un **Random Controller**, in cui è inserita un richiesta HTTP di risorse. Il **Constant Throughput Timer** permette di regolare il numero di richieste da effetturare al minuto. Al fine di ottenere dei report esplicativi dell'esecuzione del test, è stato utilizzato un **Simple Data Writer**.
Per le informazionia basso livello (analisi del bottleneck di prestazioni) è stato utilizzato il tool **vmstat**, fatto eseguire sul server.
Infine i dati ottenuti vengono processati attraverso uno script Python.

2.3. Grafici e analisi

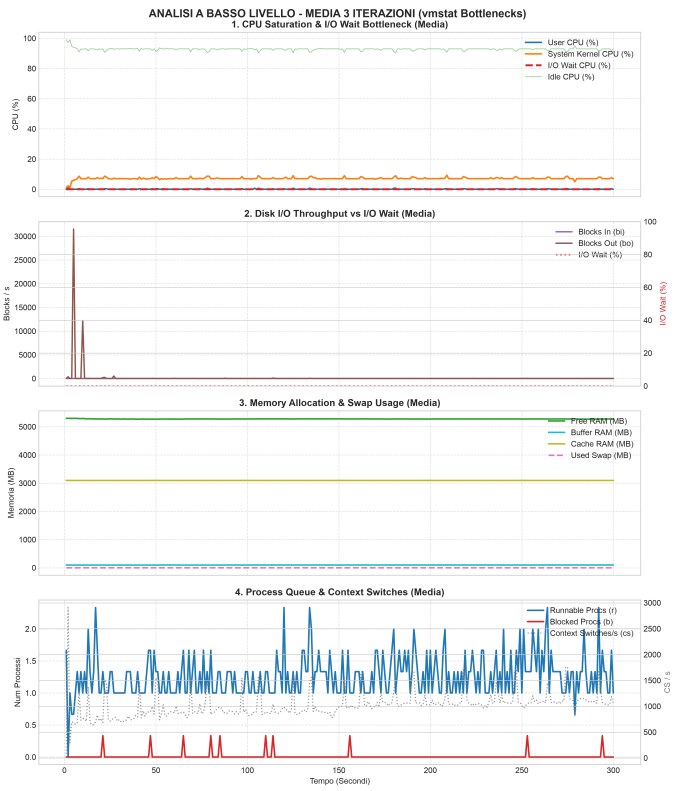


Eseguito lo script, sono stati ottentui i seguenti grafici, che rappresentano rispettivamente il **throughput**, ovvero il tasso di richieste che il sistema ha potuto servire, il **response time**, ovvero la media dei tempi di riposta delle richieste per quel carico, e la **power**, che è un valore riassuntivo dei precedenti due parametri, ovvero

$$\text{Power} = \frac{\text{Throughput}}{\text{Response Time}}. \tag{1}$$

Analizzando tale grafico si può notare che:

- Il Throughput crolla rapidamente da 8 a ∼1.5 req/sec.
- Il Response Time esplode fino a 20-50 secondi.
- Il Power crolla a zero nei primi secondi: il sistema perde totalmnete la sua efficienza di elaborazione.



Analisi dei bottleneck Analizzando i dati a basso livello, si vede che:

- La CPU è quasi totalmente in attiva (∼92
- La RAM libera è maggiore di 5GB.
- Il Disk I/O e I/O Wait sono piatti dopo il picco iniziale.

Questo fa pensare che il collo di bottiglia non è di tipo hardware.
Si tratta invece di un bottleneck di configurazione software: i worker/thread di Apache sono saturi per via dei file grandi in download, mettendo le nuove richieste in coda senza che la CPU debba effettivamente lavorare.

2.4. Analisi della Fairness e della Ripetibilità del Benchmark

Per valutare l'equità nella ripartizione delle risorse di rete e la stabilità complessiva dei test di carico, è stato adottato l'**Indice di Fairness di Jain (Jain's Fairness Index)**.
La valutazione è stata condotta su due livelli: inter-iterazione (per verificare la riproducibilità e determinismo del test) e intra-iterazione (per rilevare eventuali fenomeni di starvation tra i thread/client concorrenti).
Dato un insieme di n entità concorrenti e definita con x_i la metrica di prestazione associata all'entità i-esima, l'indice Jain J è calcolato come:

$$J(x_1, x_2, \dots, x_n) = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2} \tag{2}$$

L'indice assume valori compresi nell'intervallo $\left[\frac{1}{n}, 1.0\right]$.
Poiché i test prevedono l'esecuzione di Thread Group Paralleli che richiedono risorse di dimensioni differenti, la metrica, x_i non misura il semplice numero di richieste al secondo, che premierebbe fittiziamente i file di piccola dimesione, bensì la **banda passante occupata** espressa in MB/s:

$$x_i = \text{Bandwidth}_i = \frac{\text{Byte Total Trasferiti}_i}{1024^2 \cdot \Delta t} \tag{3}$$

L'indice di Jain inter-iterazione calcolato sia sul Throughput (0.9965) sia sulla Bandwidth (0.9968) è eccezionalmente prossimo al valore ideale di 1.0. Questo risultato attesta che il setup di test è perfettamente deterministico, stabile e riproducibile. Non sono state rilevate interferenze esterne (es. rumore di rete, processi di background sulla VM) che abbiano alterato i risultati tra un'esecuzione e l'altra.
Analizzando i dati all'interno della singola iterazione, l'indice di Jain calcolato sui thread scende sotto la soglia ottimale (0.90), oscillando tra 0.7441 e 0.8163. Un valore di fairness significativamente inferiore a 1.0 all'interno della singola esecuzione dimostra l'insorgere di Thread Starvation. Sotto il carico massimo di 50 utenti concorrenti, le risorse di elaborazione del server web non vengono distribuite in modo uniforme: alcuni thread riescono ad accaparrarsi i socket di rete e completare più richieste, mentre altri rimangono bloccati nelle code del server per tempi prolungati (spiegando i tempi di risposta massimi fino a 200+ secondi registrati da JMeter).

3. PCA E CLUSTERING

3.1. Obiettivo e Setup

L'obiettivo è quello di effettuare una **workload characterization** completa: generando un workload reale su un Web Server, si effettueranno misurazioni HL e LL, applicando PCA e clustering su entrambe. Si effettua inoltre la costruzione di un workload sintetico e si effettua un confronto statistico tra i due carichi. Per effettuare ciò viene utilizzate la macchina già precedentemente descritta nella tabella 2.

3.2. Raccolta dati

3.2.1. *Generazione dei file campione*

Per la generazione dei file campione, viene utilizzata la logica delle 5 categorie di taglia del documento (Low, Mid-Low, Mid, Mid-High, High), ma con dati casuali piuttosto che file reali multimediali.

Per semplicità, viene istanziato sulla stessa macchina un Web Server, utilizzando `FastAPI`.

3.2.2. *Generazione del traffico e misurazione*

Una parte fondamentale ora è la raccolta di dati statistici durante l'esecuzione delle richieste al Server. Utilizzando file statici, vengono effettuate delle richieste al server dei file campione.

- **Load Generator**: genera traffico HTTP concorrente verso le risorse, con un numero configurabile di thread worker e un ramp—up graduale (i thread partono scaglionati nel tempo, non tutti insieme). Le richieste sono pesate per categoria dimensionale (`DEFAULT_TIER_WEIGHTS`), in modo da avere un traffico quanto più realistico possibile.
- **Collector**: basato su `psutil`, raccoglie due tipi di metriche a intervalli regolari
  - globali di sistema (CPU, memoria, swap, I/O disco/rete — queste ultime come snapshot delta tra snapshot successivi, non contatori cumulativi da boot, altrimenti la PCA noterebbe solo il tempo che scorre)
  - di processo, specifiche del server (CPU%, RSS, thread attivi, file descriptor aperti)

Tabella 3. Ripartizione dei File per Fascia di Risorse (`RESOURCE_TIERS`)

Categoria	Parametro	Valore
low	File 1	13,5 KB
	File 2	20 KB
	File 3	23,4 KB
	File 4	98 KB
	File 5	99,7 KB
mid_low	File 1	137 KB
	File 2	139 KB
	File 3	184 KB
	File 4	458 KB
	File 5	514 KB
mid	File 1	716 KB
	File 2	0,98 MB
	File 3	0,99 MB
	File 4	1 MB
	File 5	1,3 MB
mid_high	File 1	2 MB
	File 2	2,13 MB
	File 3	2,14 MB
	File 4	5,07 MB
	File 5	5,19 MB
high	File 1	9,21 MB
	File 2	9,38 MB
	File 3	9,58 MB
	File 4	10,1 MB

Note: Dati estratti dal dizionario Python `RESOURCE_TIERS`.

- **Orchestrazione**: i due processi (collector e load generator) vengono avviati in parallelo, non in sequenza, coordinati da uno script dedicato; infatti è essenziale che si sovrappongano temporalmente, poiché il `Collector` deve effettuare le misurazioni durante la generazione del carico. Tutti i thread worker sono attivi simultaneamente per l'intera durata del test.

3.3. Pulizia e caratterizzazione HL/LL (reale)

3.3.1. *Pulizia del dataset*

Prima di procedere con PCA e clustering, ciascun dataset (HL, LL globale, LL di processo) viene sottoposto a un'analisi preliminare delle distribuzioni campionarie. Per ogni colonna vengono calcolate la media, la deviazione standard e i quartili; inoltre, viene generato un istogramma affiancato da un boxplot al fine di individuare colonne poco significative o affette da outlier. In particolare, vengono identificate come candidate alla rimozione le colonne che presentano una cardinalitá di valori unici estremamente bassa rispetto al numero di osservazioni (soglia arbitraria dell'1%), e le righe che si discostano dalla media di oltre 4 deviazioni standard su almeno una colonna. É importante sottolineare che questi criteri automatici forniscono solamente un *segnale*: la decisione finale di rimozione viene presa dopo l'ispezione manuale, per evitare di eliminare varianza informativa (ad esempio, colonne con pochi valori disitinti ma comunque rilevanti, come `swap_percent`).

Dataset LL gloabale. La colonna `mem_total` risulta costante per l'intera durata della misurazione e viene pertanto rimossa. Le colonne relative allo swap (`swap_used`, `swap_free`, `swap_percent`) presentano una bassa cardinalitá di valori, dovuta dal fatto che il gestore di memoria del sistema operativo le aggiorna a blocchi discreti piuttosto che con continuitá; tuttavia mostrando un range di variazione significativo, vengono mantenute. Dall'analisi degli outlier emerge un pattern ricorrente: un blocco di righe consecutive all'inizio della misurazione (transiente di *ramp-up*, corrispondente all'avvio graduale dei thread worker) e un blocco analogo al termine (transiente di *coll-down*, dovuto alla chiusura delle connessioni e alla fase di rientro del sistema a uno stato di riposo). Tali righe vengono rimosse, per un totale di 32 righe iniziali e 23 finali.

Tabella 4. Sintesi delle operazioni di pulizia applicate ai dataset reali

Dataset	Righe (grezze → pulite)	Colonne rimosse	Trimming
LL globale	3615 → 3560	`mem_total`	32 / 23
LL processo	3615 → 3560	`pid`, I/O	32 / 23
HL	679046 → 679046	`responseCode`, `success`, `URL`	—

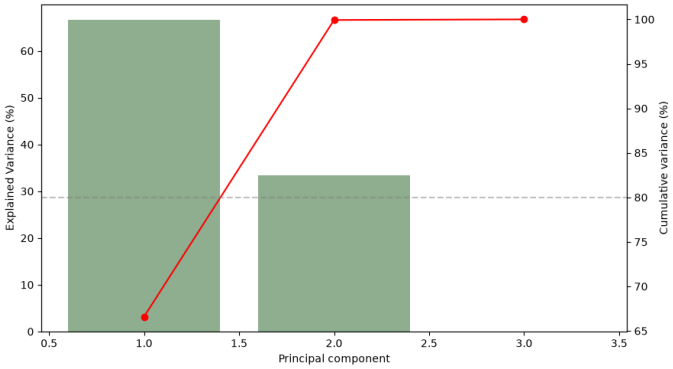


Figura 2. Scree plot delle componenti principali — LL globale (reale).

Dataset LL di processo. . Vengono rimosse le colonne `pid` (costante) e `proc_io_read_bytes`/`proc_io_write_bytes`, non disponibili per la piattaforma utilizzata per la raccolta dati. Analogamente al caso precedente, vengono rimosse 32 righe iniziali e 23 finali corrispondenti ai transienti di avvio e chiusura.

Dataset HL. Vengono rimosse le colonne `responseCode` e `success` (costanti poiché tutte le richieste hanno avuto esito positivo) e `URL` (ridondante rispetto a `label`). A differenza dei dataset LL, non viene applicato alcun trimming per indice: gli outlier individuati (circa lo 0.3% delle osservazioni) risultano distribuiti lungo l'intera durata della misurazione, non concentrati in un transiente iniziale o finale, e vengono pertanto interpretati come variabilitá fisiologica del sistema sotto carico piuttosto che errori di misura.

3.3.2. *Analisi delle componenti principali (PCA)*

L'analisi PCA viene condotta standardizzando preventivamente le feature (essendo le colonne espresse su scale eterogenee: percentuali, byte, conteggi), per poi analizzare la varianza spiegata cumulatica sa ciascuna componente.

Per il dataset LL globale, si osserva che le prime 3 componenti principali spiegano oltre l'80% della varianza totale, coerentemente con l'attesa correlazione tra le metriuche di memoria (`mem_used`, `mem_available`, `mem_free`) e quelle di I/O.

3.3.3. *Clustering gerarchico e selezione dei parametri*

Per determinare la combinazione ottimale di numero di componenti principali e numero di cluster, viene condotto uno sweep sistematico: per ciascuna coppia (numero di compoenti, numero di cluster) viene applicato un clustering gerarchico agglomerativo (metodo di Ward), calcolando la devianza intra-cluster, inter-cluster e la devianza totale persa.

Sulla base di questo criterio, per il dataset LL globale viene scelta la combinazione a **3 compoenti e 15 cluster**; per il dataset LL di processo, la combinazione a **2 componenti e 15 cluster** (devianza persa pari a 0.0609).

Tabella 5. Sweep delle combinazioni PCA + Clustering — LL globale (reale)

N. componenti	N. cluster	Devianza intra	Devianza persa
3	15	0.0155	**0.0566**
3	10	0.0253	0.0965
3	5	0.0706	0.2110

Nota: tabella parziale, riportate le configurazioni più significative. La combinazione a 3 componenti e 15 cluster minimizza la devianza persa.

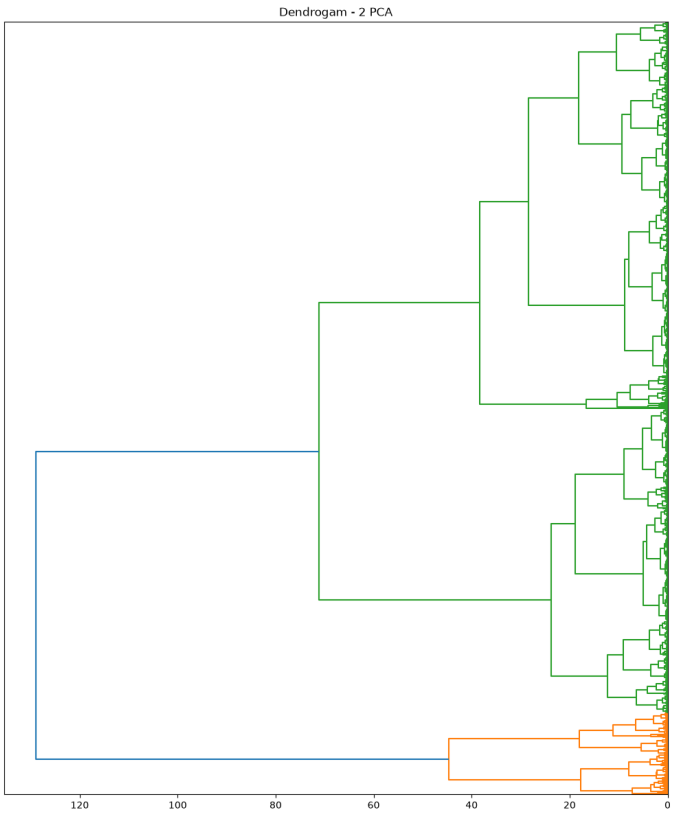


Figura 3. Dendrogramma — LL globale, 3 componenti, 15 cluster (reale).

3.4. Clustering e scalabilità

3.4.1. *Il problema della scalabilità*

A differenza dei dataset LL (dell'ordine di poche migliaia di osservazioni), il dataset HL raccolto durante la sollecitazione reale conta 679 046 righe. L'approccio di clustering gerarchico agglomerativo utilizzato per i dataset LL richiede il calcolo della matrice delle distanze tra tutte le coppie di osservazioni, con complessità $O(n^2)$ sia in tempo che in memoria: per $n = 679\,046$, ciò comporterebbe circa 2.3×10^{11} coppie, corrispondenti a diverse centinaia di gigabyte di memoria — un requisito non soddisfacibile dall'hardware utilizzato (Tabella 2).

Si rende quindi necessario un approccio alternativo, in grado di elaborare l'intero dataset senza mai caricarlo integralmente in memoria né calcolarne la matrice delle distanze complete.

3.4.2. *Pipeline incrementale a blocchi (chunking)*

La soluzione adottata scompone il dataset in blocchi (*chunk*) di dimensione fissa, elaborati sequenzialmente in più passate sul file sorgente. La pipeline si articola in cinque fasi:

1. **Standardizzazione incrementale**: le statistiche di media e deviazione standard necessarie alla standardizzazione delle feature vengono calcolate incrementalmente su blocchi successivi, tramite un accumulatore (`StandardScaler.partial_fit`), senza mai richiedere l'intero dataset in memoria.
2. **PCA incrementale**: analogamente, la riduzione dimensionale viene effettuata tramite `IncrementalPCA`, che aggiorna le componenti principali blocco per blocco anziché tramite un'unica decomposizione in valori singolari sull'intera matrice.
3. **Costruzione del CF-tree (Birch)**: le osservazioni proiettate nello spazio delle componenti principali vengono aggregate incrementalmente in una struttura ad albero compatta (*Clustering Feature tree*), secondo l'algoritmo BIRCH [1], che riassume porzioni del dataset in *sub-cluster* rappresentati dal proprio centroide e dal numero di osservazioni associate.
4. **Clustering gerarchico sui sub-cluster**: poiché il numero di sub-cluster generati da BIRCH è ordini di grandezza inferiore al numero di osservazioni originali, diventa possibile applicare il clustering gerarchico agglomerativo (identico a quello utilizzato per i dataset LL) direttamente sui centroidi dei sub-cluster.
5. **Assegnazione finale**: in un'ultima passata sul dataset, ogni osservazione originale viene associata al proprio sub-cluster e, transitivamente, al cluster finale risultante dal passo precedente.

Questo approccio consente di derivare una caratterizzazione dell'intero dataset — non di un suo campione — pur mantenendo un costo computazionale gestibile, poiché il passaggio quadratico viene applicato non alle 679 046 osservazioni originali, ma ai soli sub-cluster generati da BIRCH.

3.4.3. *Risultati*

Applicando la pipeline al dataset HL (feature `elapsed`, `bytes`, `latency`), le prime 2 componenti principali spiegano il 99.93% della varianza totale, un valore più elevato rispetto ai dataset LL, coerentemente con la forte correlazione attesa tra tempo di risposta, byte trasferiti e latenza. La costruzione del CF-tree con soglia threshold $= 0.5$ produce 153 sub-cluster, sui quali viene successivamente applicato il clustering gerarchico per ottenere 15 cluster finali, in linea con il numero di cluster scelto per i dataset LL. La devianza persa complessiva risulta pari a 0.0106.

Tabella 6. Risultati della pipeline incrementale — dataset HL (reale)

N. componenti	N. cluster	N. sub-cluster (BIRCH)	Devianza persa
2	15	153	0.0106

3.4.4. Caratterizzazione del workload sintetico tramite tabella di contingenza

A partire dall'assegnazione di cluster ottenuta per ciascuna delle 679 046 richieste, viene costruita una tabella di contingenza tra cluster e risorsa richiesta (`label`), al fine di identificare, per ciascun cluster, la risorsa più frequentemente richiesta. Dall'analisi emerge una distribuzione fortemente sbilanciata tra i cluster: i primi quattro cluster per dimensione coprono congiuntamente oltre il 99% delle osservazioni, mentre i restanti undici cluster contano da 1 a poche centinaia di osservazioni ciascuno. Tale sbilanciamento è coerente con la natura del CF-tree di BIRCH, che tende a isolare in sub-cluster di piccole dimensioni le osservazioni con comportamento anomalo (ad esempio dovuto a variabilità di rete), mentre concentra la maggioranza del traffico "regolare" in pochi cluster densi. I cluster con dimensione inferiore a una soglia minima (50 osservazioni) vengono pertanto esclusi dalla costruzione del workload sintetico, poiché rappresentano una frazione trascurabile del traffico complessivo (0.014%) e sono verosimilmente riconducibili a rumore statistico piuttosto che a pattern di utilizzo genuini.

3.5. Costruzione del workload sintetico

3.5.1. Pesatura delle risorse

A partire dalla tabella di contingenza cluster–risorsa descritta nella sezione precedente, e filtrati i cluster di dimensione inferiore alla soglia minima, si procede alla costruzione del workload sintetico. Per ciascun cluster sopravvissuto al filtraggio viene identificata la risorsa più frequentemente richiesta; qualora la medesima risorsa risulti la più frequente in più cluster distinti, le rispettive dimensioni vengono sommate, ottenendo un peso complessivo proporzionale alla frequenza osservata nel workload reale.

Il peso assegnato a ciascuna risorsa viene quindi normalizzato su un budget complessivo di 100 unità, con un vincolo di peso minimo pari a 1 per evitare che risorse comunque rilevanti, ma con frequenza relativa inferiore all'1%, vengano eliminate dall'arrotondamento.

Tabella 7. Pool pesata delle risorse per il workload sintetico

Risorsa	Dimensione cluster aggregata	Peso (%)
`Sample_13.5KB.bin`	485,444	71
`Sample_99.7KB.bin`	153,223	23
`Sample_9.21MB.bin`	33,547	5
`Sample_5.07MB.bin`	6,367	1
`Sample_98KB.bin`	372	1

Nota: i pesi relativi a `Sample_5.07MB.bin` e `Sample_98KB.bin` sono stati vincolati al valore minimo di 1 dall'arrotondamento, pur rappresentando rispettivamente circa lo 0.94% e lo 0.05% del traffico osservato.

A differenza del workload reale, in cui le richieste sono distribuite su 24 risorse con pesi decrescenti per categoria dimensionale, il workload sintetico risulta cosí concentrato su un numero ridotto di risorse (5), fortemente sbilanciato verso la risorsa di dimensione minore, che da sola rappresenta il 71% del traffico generato. Tale concentrazione riflette direttamente la distribuzione osservata nei cluster del dataset HL reale.

3.5.2. Generazione del traffico sintetico

Il workload sintetico viene sollecitato sul medesimo Web Server, riutilizzando l'infrastruttura di raccolta dati (load generator e collector) già descritta, con la sola differenza che il load generator seleziona le risorse dalla pool pesata per cluster anziché da quella pesata per categoria dimensionale.

La forte concentrazione del traffico su una singola risorsa di piccola dimensione (13.5 KB), unita all'elevato numero di richieste completabili nell'unità di tempo, ha reso necessario introdurre un tempo di attesa (*think time*) di 10 ms tra una richiesta e la successiva per ciascun thread worker. In assenza di tale accorgimento, il throughput generato eccede la capacità del sistema operativo di gestire nuove connessioni di rete nell'unità di tempo (esaurimento dei *file descriptor* disponibili), causando un tasso di fallimento delle richieste non rappresentativo di un carico applicativo realistico, bensì di una condizione di saturazione del client di generazione del traffico.

Con tale accorgimento, la sollecitazione del workload sintetico (durata 3600 secondi, 15 thread worker) produce 1 224 394 richieste, con un tasso di successo del 100%.

3.6. Caratterizzazione LL (sintetico)

A seguito della sollecitazione del workload sintetico, vengono raccolti i report a basso livello $LL'_{globale}$ e $LL'_{processo}$, secondo le medesime modalità impiegate per il workload reale. Pulizia e caratterizzazione vengono condotte applicando la medesima metodologia descritta nelle sezioni precedenti, mantenendo invariati numero di componenti principali e numero di cluster rispetto al corrispettivo dataset reale, condizione necessaria per rendere valido il successivo confronto statistico.

3.6.1. Pulizia

L'ispezione delle distribuzioni campionarie conferma, anche per i dataset sintetici, la presenza di transienti di *ramp-up* e *cool-down* analoghi a quelli osservati nei dataset reali, sebbene di estensione leggermente diversa: 11 righe iniziali e 24 finali per $LL'_{globale}$, 9 righe iniziali e 20 finali per $LL'_{processo}$.

Si osserva inoltre, per il dataset $LL'_{globale}$, un numero di outlier interni al periodo di misurazione (3.40% delle osservazioni) superiore rispetto al corrispondente dataset reale. Tale differenza è coerente con la natura del workload sintetico: la forte concentrazione del traffico su risorse di dimensione molto eterogenea (da 13.5 KB a 9.21 MB) genera transienti di carico più marcati e ricorrenti, in particolare in corrispondenza delle richieste verso le risorse di dimensione maggiore. Tali osservazioni, non concentrate in un singolo intervallo temporale ma distribuite lungo l'intera durata della misurazione, vengono interpretate come variabilità legittima del workload sintetico e pertanto mantenute nel dataset.

Tabella 8. Sintesi delle operazioni di pulizia — dataset sintetici

Dataset	Righe (grezze → pulite)	Colonne rimosse	Trimming
$LL'_{globale}$	3615 → 3580	`mem\_total`	11 / 24
$LL'_{processo}$	3615 → 3586	`pid`, I/O	9 / 20

3.6.2. PCA e Clustering

Applicando PCA e clustering gerarchico con i medesimi parametri selezionati per il workload reale (3 componenti e 15 cluster per $LL'_{globale}$; 2 componenti e 15 cluster per $LL'_{processo}$), si ottengono i risultati riportati in Tabella 9.

Tabella 9. Confronto della devianza persa tra workload reale e sintetico

Dataset	N. componenti / cluster	Devianza persa (reale)	Devianza persa (sintetico)
LL globale	3 / 15	0.0566	0.0214
LL processo	2 / 15	0.0609	0.0400

Nota: in entrambi i casi, la devianza persa risulta inferiore nel workload sintetico rispetto al reale.

In entrambi i dataset, la devianza persa risulta inferiore nel workload sintetico rispetto al corrispettivo reale. Tale risultato è atteso: essendo il workload sintetico costruito a partire da un numero ridotto di risorse (5 anziché 24), il sistema sotto sollecitazione presenta un

comportamento meno eterogeneo, e i cluster risultanti sono pertanto più compatti e meglio separati.

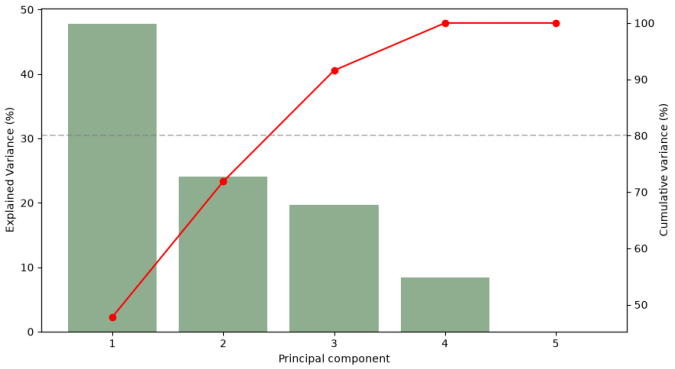


Figura 4. Scree plot delle componenti principali — LL globale (sintetico).

3.7. Validazione Statistica

3.7.1. Estrazione dei campioni per cluster

Al fine di confrontare statisticamente il workload reale e quello sintetico, per ciascun dataset (LL globale e LL di processo, nelle rispettive versioni reale e sintetica) viene estratto un campione rappresentativo per ciascuno dei 15 cluster individuati, selezionando l'osservazione più vicina al centroide del proprio cluster nello spazio delle componenti principali. Tale criterio, tra quelli possibili (selezione casuale, centroide stesso, osservazione più vicina al centroide), viene scelto in quanto privilegia elementi realmente rappresentativi del comportamento tipico di ciascun cluster, coerentemente con l'obiettivo di confrontare le caratteristiche tipiche dei due workload.

Si ottengono così due coppie di campioni ridotti, ciascuna composta da 15 osservazioni (una per cluster) nello spazio delle rispettive componenti principali: una coppia a 3 componenti per il confronto LL globale, una coppia a 2 componenti per il confronto LL di processo.

3.7.2. Test di normalità

Viene preliminarmente valutata la normalità delle distribuzioni tramite test di Kolmogorov–Smirnov, con ipotesi nulla H_0: il campione proviene da una distribuzione normale standard.

Tabella 10. Test di Kolmogorov–Smirnov sulle componenti principali

Dataset	Componente	Statistica KS	p-value	Rigetta H_0
LL globale	PC1	0.1753	0.6828	No
LL globale	PC2	0.1875	0.6022	No
LL globale	PC3	0.2733	0.1759	No
LL processo	PC1	0.1213	0.9607	No
LL processo	PC2	0.1019	0.9931	No

Nota: risultati identici tra campione reale e sintetico in ciascuna componente.

Nel caso in questione nessuna componente rigetta H_0. Tale esito va tuttavia interpretato con cautela: la dimensione campionaria estremamente ridotta ($n = 15$, una osservazione per cluster) conferisce al test una potenza statistica limitata, insufficiente a discriminare con sicurezza tra distribuzioni normali e non normali. Il mancato rigetto non costituisce pertanto una prova di normalità, ma l'assenza di evidenza sufficiente contro di essa a questa dimensione campionaria.

Per tale motivo, si procede comunque con un test non parametrico, più robusto rispetto ad assunzioni di normalità non verificabili con certezza sul campione a disposizione.

3.7.3. Test di Wilcoxon

Viene applicato il test di Wilcoxon (*ranksum*), con ipotesi nulla H_0: i due campioni (reale e sintetico) provengono da distribuzioni con la medesima mediana, componente per componente.

Tabella 11. Test di Wilcoxon ranksum — confronto reale vs. sintetico

Dataset	Componente	Statistica	p-value	Rigetta H_0
LL globale	PC1	−1.1406	0.2540	No
LL globale	PC2	1.8043	0.0712	No
LL globale	PC3	0.8503	0.3952	No
LL processo	PC1	0.4355	0.6632	No
LL processo	PC2	1.0577	0.2902	No

In nessuna delle componenti analizzate, per entrambi i dataset, il test rigetta l'ipotesi nulla a un livello di significatività del 5%. Si può pertanto concludere che non vi è evidenza statistica di una differenza tra le mediane dei workload reale e sintetico, nello spazio delle componenti principali considerate.

3.8. Conclusioni

Il presente capitolo ha descritto la costruzione di una pipeline completa per la caratterizzazione di un workload applicativo, articolata secondo lo schema a tre fasi: caratterizzazione del workload reale, costruzione e sollecitazione di un workload sintetico, validazione statistica della loro equivalenza.

L'intera pipeline è stata implementata in Python, con l'obiettivo di ottenere uno strumento modulare e riproducibile. Tale scelta ha richiesto di affrontare, rispetto all'approccio originale, alcune problematiche specifiche: in primo luogo, la scalabilità del clustering gerarchico rispetto alla dimensione del dataset HL (679 046 osservazioni), risolta tramite una pipeline incrementale a blocchi basata sull'algoritmo BIRCH; in secondo luogo, la gestione di un ambiente di raccolta dati su macchina singola, condivisa tra generazione del carico e misurazione delle risorse di sistema, che ha richiesto accorgimenti specifici (tempo di attesa tra le richieste, gestione dei limiti di sistema) per evitare che il processo di misurazione stesso alterasse il fenomeno osservato.

I risultati ottenuti mostrano che, per entrambi i dataset a basso livello analizzati (metriche globali di sistema e metriche del processo server), il workload sintetico costruito a partire dalla caratterizzazione del traffico applicativo (HL) non presenta differenze statisticamente significative rispetto al workload reale, nello spazio delle rispettive componenti principali. Tale risultato conferma, nei limiti del test di Wilcoxon applicato a un campione di ridotta numerosità, la validità dell'approccio PCA + Clustering come metodologia per la sintesi di workload rappresentativi a partire da misurazioni applicative ad alto livello.

Un limite dell'approccio adottato risiede proprio nella dimensione del campione utilizzato per la validazione statistica finale ($n = 15$, pari al numero di cluster), che ne limita la potenza statistica. Un possibile sviluppo futuro potrebbe prevedere l'estrazione di più osservazioni rappresentative per cluster, o l'impiego di test statistici progettati per campioni di dimensione ridotta, al fine di rafforzare la robustezza della conclusione raggiunta.

4. WORKLOAD CHARACTERIZATION

4.1. Obiettivo e Setup

L'analisi e la caratterizzazione del carico di lavoro (*Workload Characterization*) costituiscono una fase fondamentale nell'ambito dell'ingegneria delle prestazioni dei sistemi di elaborazione. Comprendere la natura statistica e la dinamica temporale delle risorse hardware e software sollecitate da un'applicazione reale rappresenta il prerequisito essenziale per la costruzione di modelli prestazionali

accurati, la pianificazione della capacità (*Capacity Planning*) e la generazione di carichi sintetici riproducibili. L'attività si articola in due macro-fasi complementari:

1. **Caratterizzazione e Modellazione del Carico Reale:** Estrazione e analisi delle metriche di sistema registrate tramite un monitor basato sulla libreria `psutil` con intervallo di campionamento pari a 1s (`report_LL_real.txt`). In questa fase vengono identificate le proprietà descrittive, le correlazioni inter-risorsa, la struttura di autocorrelazione temporale e la distribuzione di probabilità teorica che meglio approssima le sollecitazioni del sistema.
2. **Progettazione e Validazione del Generatore Sintetico:** Sviluppo di uno script software in ambiente Python in grado di emulare il comportamento del carico reale basandosi sul campionamento stocastico dalla distribuzione appresa. Il carico sintetico generato viene tracciato analogamente al sistema reale (`report_LL_synthetic.txt`) e sottoposto a una procedura rigorosa di validazione quantitativa (test di Goodness-of-Fit e confronto delle CDF empiriche).

Dall'osservazione preliminare dei dati di tracciamento, il carico di lavoro oggetto di studio si caratterizza per una spiccata natura **altamente concorrente e interattiva**:

- **Elevata Frequenza di Context Switching:** Il sistema registra una frequenza media di cambi di contesto (`cs`) nell'ordine dei 26.000 cs/s, con picchi intensi che superano la soglia degli 80.000 cs/s.
- **Prevalenza dell'Overhead di Kernel:** Il consumo di CPU in modalità sistema (`sy`) e utente (`us`) riflette un carico guidato principalmente da operazioni di sincronizzazione, I/O in memoria e commutazioni di stato gestite dallo scheduler del Kernel.
- **Assenza di Bottleneck su Disco:** Il tempo di attesa per operazioni di I/O (`wa`) si attesta su valori trascurabili o prossimi allo zero, confermando che il collo di bottiglia principale del sistema è legato alla gestione della concorrenza e dei processi piuttosto che alle periferiche di memorizzazione di massa.

L'obiettivo finale risiede nel realizzare un generatore sintetico che sappia riprodurre fedelmente la risposta del sistema target, bilanciando accuratamente la generazione di *Context Switches* reali tramite chiamate IPC e l'occupazione della CPU.

4.2. Ingestione Dati e Pre-processing

La fase iniziale del flusso di lavoro analitico consiste nell'ingestione, pulizia e strutturazione dei tracciamenti di sistema. I dati grezzi, acquisiti tramite l'utility standard di monitoraggio `vmstat`, richiedono una specifica procedura di parsing per trasformare l'output di testo non strutturato in una struttura dati tabellare adatta all'analisi statistica e alla modellazione probabilistica. I file di log prodotti dal sistema (`report_LL_real.txt` per il carico reale e `report_LL_synthetic.txt` per quello sintetico) contengono campionamenti eseguiti a frequenza unitaria (1s). Ogni riga del log rappresenta uno stato istantaneo delle risorse del sistema operativo articolato su 17 metriche standard:

- **Procs (`r`, `b`):** Numero di processi in attesa di tempo di esecuzione o in stato di sleep non interrompibile.
- **Memory (`swpd`, `free`, `buff`, `cache`):** Memoria virtuale utilizzata, memoria libera, e memoria allocata per buffer e cache di I/O (espresse in KiB).
- **Swap (`si`, `so`):** Ammontare di memoria scambiata da e verso il disco al secondo.
- **IO (`bi`, `bo`):** Blocco di ingressi ed uscite da/verso dispositivi a blocchi.
- **System (`in`, `cs`):** Frequenza di interrupt hardware e *Context Switches* al secondo.
- **CPU (`us`, `sy`, `id`, `wa`, `st`):** Ripartizione percentuale del tempo CPU tra User Mode, System Mode, Idle, I/O Wait e Stolen Time.

Tabella 12. Statistiche descrittive delle metriche primarie del carico reale ($N = 897$).

Metrica	Media	Dev. Std.	Minimo	Mediana	Massimo
`cs` [switch/s]	26 057, 26	7 542, 18	4 120, 00	26 210, 00	83 441, 00
`us` [%]	10, 42	4, 15	1, 00	10, 00	28, 00
`sy` [%]	5, 81	2, 38	1, 00	6, 00	18, 00
`free` [KiB]	88 250, 34	12 450, 12	62 120, 00	87 900, 00	115 400, 00

La pipeline di pulizia esegue i seguenti passaggi sequenziali:

1. **Filtraggio dell'Header e del Rumore di Formattazione:** L'output del monitor ripete periodicamente le righe di intestazione (`procs`, `swpd`, `r b`). Lo script scarta le righe non contenenti campionamenti numerici o aventi un numero di token inferiore a 17.
2. **Normalizzazione dei Separatori Decimali:** Per garantire la compatibilità con le convenzioni di formattazione locali, le eventuali virgole utilizzate come separatori decimali vengono convertite sistematicamente nel punto decimale standard (`.`).
3. **Cast Tipologico e Gestione dei Valori Nulli:** I vettori di stringhe estratti vengono convertiti in tipi numerici ad alta precisione (`float64`). Eventuali anomalie di parsing vengono gestite forzando i valori non validi a `NaN` ed eliminandoli.

4.3. Caratterizzazione descrittiva del Carico Reale

A seguito della pipeline di ingestione e pulizia dei dati, è stata condotta un'analisi statistica descrittiva dettagliata sul tracciamento reale (`report_LL_real.txt`). L'analisi comprende 897 campioni temporali consecutivi (corrispondenti a circa 15 minuti di osservazione continua a intervalli di 1s) e si focalizza sugli indicatori chiave di prestazione (*Key Performance Indicators* — KPI) per identificare l'impronta prestazionale del sistema. Le metriche fondamentali estratte per caratterizzare la sollecitazione dell'hardware e del sistema operativo sono:

- **Context Switches (`cs`):** Frequenza di avvicendamento dei processi/thread sulla CPU al secondo. Rappresenta la variabile guida dell'intero workload.
- **CPU User Mode (`us`):** Percentuale di tempo CPU dedicata all'esecuzione del codice applicativo nello spazio utente.
- **CPU System Mode (`sy`):** Percentuale di tempo CPU spesa nell'esecuzione di codice del Kernel (gestione interrupt, chiamate di sistema, context switch, scheduling).
- **Free Memory (`free`):** Ammontare di memoria RAM fisica immediatamente disponibile per il sistema (espressa in KiB).

La Tabella 12 riporta i principali indici di posizione e dispersione (media, deviazione standard, minimo, mediana e massimo) calcolati sui campioni del carico reale.

Dall'analisi dei dati descrittivi emergono le seguenti proprietà strutturali del carico in esame:

1. **Rapporto CPU User/System ($us/sy \approx 1,79$):** Il sistema spende circa il $10,42\%$ del tempo in User Mode e il $5,81\%$ in System Mode, per un consumo totale di CPU medio pari a circa il $16,23\%$. La quota rilevante di System Mode conferma che la gestione dell'I/O e della concorrenza genera un overhead di kernel non trascurabile rispetto al calcolo applicativo puro.
2. **Variabilità dei Context Switches:** La frequenza dei cambi di contesto evidenzia una forte dinamicità: a fronte di un valore medio di $\sim$ 26.057 cs/s, si registrano picchi impulsivi fino a 83.441 cs/s. Tale ampiezza di intervallo suggerisce l'esistenza di fasi a raffica (*bursty behavior*) tipiche di ambienti con thread altamente interattivi.
3. **Inesistenza di I/O Bottleneck (`wa` $\approx 0\%$):** La percentuale di tempo CPU spesa in attesa di operazioni di I/O su disco (`wa`) si

Tabella 13. Matrice di correlazione lineare di Pearson tra i KPI del carico reale.

	cs	us	sy	free
cs	1,0000	−0,1996	−0,0995	0,3250
us	−0,1996	1,0000	0,8893	−0,1690
sy	−0,0995	0,8893	1,0000	−0,1722
free	0,3250	−0,1690	−0,1722	1,0000

attesta costantemente su valori prossimi allo zero. Ciò consente di classificare formalmente il workload come **In-Memory / CPU-bound con elevata concorrenza**, escludendo il sottosistema di memorizzazione secondaria dai colli di bottiglia del sistema.

4.4. Modellazione Statistica Avanzata

Per catturare la struttura stocastica sottostante del carico e guidare la sintesi del workload generato, l'analisi descrittiva è stata integrata da una modellazione probabilistica avanzata. Tale fase comprende lo studio delle dipendenze lineari tra le metriche di sistema, l'analisi della memoria temporale della serie storica e l'individuazione della distribuzione teorica ottimale tramite test di bontà dell'adattamento.

4.4.1. Analisi delle Correlazioni Inter-Risorsa

Per comprendere l'accoppiamento funzionale tra i diversi sottosistemi hardware (CPU, memoria e scheduler), è stata calcolata la matrice di correlazione di Pearson tra i principali indicatori di prestazione ($N = 897$). Il coefficiente di correlazione lineare $r_{x,y}$ tra due generiche variabili x e y è definito formalmente come:

$$r_{x,y} = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{N}(x_i - \bar{x})^2 \sum_{i=1}^{N}(y_i - \bar{y})^2}}$$

La Tabella 13 riporta la matrice di correlazione risultante per i KPI esaminati.

Dalla matrice emergono due evidenze rilevanti per la progettazione del generatore:

1. **Forte accoppiamento lineare CPU User / System ($r_{us,sy} = 0,8893$):** L'attività applicativa in User Mode e l'attività del Kernel in System Mode sono legate da una fortissima correlazione positiva. All'aumentare delle richieste di calcolo dell'applicazione utente, il Kernel risponde incrementando in modo quasi proporzionale le operazioni di gestione della memoria e dello scheduling.
2. **Indipendenza relativa dei Context Switches dalla CPU ($r_{cs,us} = -0,1996$, $r_{cs,sy} = -0,0995$):** La frequenza dei cambi di contesto presenta una correlazione minima (o leggermente negativa) con l'occupazione della CPU. Ciò dimostra che l'elevata frequenza di `cs` non è causata da saturazione di calcolo, bensì da primitive di sincronizzazione (es. lock contention, wait/notify) e commutazioni tra thread prevalentemente in-memory.

4.4.2. Autocorrelazione Temporale

Per valutare la persistenza temporale della serie dei Context Switches e modellare l'evoluzione dinamica del carico nel tempo, è stata analizzata la funzione di autocorrelazione (*Autocorrelation Function* - ACF). Il coefficiente di autocorrelazione al ritardo temporale k (r_k) è calcolato come:

$$r_k = \frac{\sum_{t=1}^{N-k}(cs_t - \bar{cs})(cs_{t+k} - \bar{cs})}{\sum_{t=1}^{N}(cs_t - \bar{cs})^2}$$

La Tabella 14 sintetizza i valori di autocorrelazione ottenuti per la variabile `cs` a diversi intervalli di ritardo temporale ($k \in \{1, 5, 10, 30\}$ s).

Tabella 14. Autocorrelazione della serie temporale dei Context Switches (`cs`).

Lag Temporale	Intervallo (k)	Autocorrelazione (r_k)
Lag-1s	1 s	0,5437
Lag-5s	5 s	0,2798
Lag-10s	10 s	0,1716
Lag-30s	30 s	0,0380

I risultati evidenziano una **struttura autoregressiva a breve termine** ($r_1 = 0,5437$), indicando che un secondo caratterizzato da un elevato numero di cambi di contesto tende a essere seguito da un secondo analogamente sollecitato. L'autocorrelazione decade progressivamente verso lo zero nell'arco di 30 s ($r_{30} = 0,0380$), confermando che su intervalli medio-lunghi il processo stocastico è stazionario e privo di trend di lungo periodo.

4.4.3. Fitting delle distribuzioni temporali e Test Goodness-of-Fit

Al fine di determinare il modello probabilistico in grado di generare la variabile target `cs`, la serie empirica dei Context Switches è stata sottoposta a un processo di fitting parametrico rispetto a quattro distribuzioni teoriche candidate: *Normale*, *Gamma*, *Lognormale* ed *Esponenziale*.

Per verificare formalmente la bontà dell'adattamento (*Goodness-of-Fit* - GoF), è stato applicato il test di **Kolmogorov-Smirnov a un campione** (K-S test). La statistica di distanza D del test misura la massima discrepanza assoluta tra la funzione di distribuzione cumulata empirica $F_e(x)$ e la funzione di distribuzione cumulata teorica $F_t(x)$:

$$D = \sup_x |F_e(x) - F_t(x)|$$

L'ipotesi nulla H_0 stabilisce che i dati empirici provengano dalla distribuzione teorica considerata. Un valore di p-value $> 0,05$ indica l'impossibilità di rifiutare H_0 al livello di significatività del 5%. La Tabella 15 riporta i risultati quantitativi del test GoF.

Tabella 15. Risultati del test di Kolmogorov-Smirnov a 1 campione sulle distribuzioni candidate.

Modello Candidato	Statistica D	p-value	Esito Test ($\alpha = 0,05$)
Gamma	**0,04353**	**0,06483**	**Accettato (H_0 non rifiutata)**
Normale	0,04665	0,03906	Rifiutato ($p < 0,05$)
Lognormale	0,04736	0,03463	Rifiutato ($p < 0,05$)
Esponenziale	0,34364	$4,32378 \times 10^{-95}$	Rifiutato ($p \ll 0,05$)

I risultati confermano rigorosamente che la **distribuzione Gamma** costituisce il miglior modello teorico per rappresentare i Context Switches, registrando la statistica di distanza minima ($D = 0,04353$) e l'unico p-value superiore alla soglia critica ($p = 0,06483 > 0,05$). Di conseguenza, i parametri stimati della distribuzione Gamma (shape, loc, scale) vengono adottati come base per il campionamento stocastico nel generatore di carico sintetico.

4.5. Architettura del generatore di Carico Sintetico

La progettazione del generatore di carico sintetico ha l'obiettivo di tradurre la caratterizzazione probabilistica appresa nella fase analitica in una sollecitazione software reale e controllabile. Il generatore deve riprodurre sia il comportamento stocastico della frequenza dei Context Switches (`cs`), sia il bilanciamento tra l'occupazione della CPU in User Mode (`us`) e System Mode (`sy`).

4.5.1. Progettazione del Meccanismo IPC

Per sollecitare lo scheduler del Kernel Linux e forzare l'esecuzione di cambi di contesto reali (anziché simulati ad alto livello), è stato

adottato un meccanismo di comunicazione inter-processo (IPC) basato su pipe anonime POSIX (`os.pipe()`).

Superamento del Global Interpreter Lock (GIL) Nelle prime iterazioni di sviluppo, l'utilizzo dell'architettura multi-threading in Python si è rivelato inadeguato: il *Global Interpreter Lock* (GIL) limita l'esecuzione parallela del bytecode Python a un singolo core CPU, introducendo un collo di bottiglia strutturale attorno ai 15.000–20.000 cs/s. Per superare tale vincolo e raggiungere la capacità di saturazione richiesta dal carico reale (fino a $\sim$ 80.000 cs/s), l'architettura è stata riprogettata sfruttando il modulo `multiprocessing`:

- **Processi Indipendenti:** Vengono create N coppie di processi worker completamente disaccoppiati nell'indirizzamento di memoria, eliminando la contesa del GIL.
- **Scambio "Ping-Pong" via Pipe:** Ciascuna coppia di processi condivide due descrittori di file (`r_fd`, `w_fd`). Un processo effettua un'operazione di scrittura bloccante (`os.write`), mentre il partner legge un byte (`os.read`). Questa sequenza forza il Kernel a sospendere il processo lettore in attesa di dati e a risvegliare lo scrittore, generando un *Context Switch* hardware effettivo e incrementando il tempo CPU in System Mode (`sy`).
- **Generazione Controllata di CPU User:** All'interno del ciclo IPC di ciascun worker è integrato un micro-ciclo di calcolo aritmetico (`dummy_calc`). Questo consente di bilanciare la percentuale di CPU User (`us`) rispetto a quella di System Mode (`sy`), emulando l'elaborazione applicativa del workload reale.

4.5.2. Sincronizzazione e Campionamento Probabilitstico

Il generatore opera secondo una logica di controllo temporale a intervalli discreti della durata di 1 s (*pacing loop*), garantendo la stazionarità e la dinamica temporale osservate nel sistema reale.

Campionamento Stocastico All'inizio di ogni secondo t, il processo principale campiona il target di Context Switches CS_t dalla distribuzione Gamma parametrizzata durante l'analisi statistica:

$$CS_t \sim \text{Gamma}(\text{shape} = \alpha, \text{loc} = \mu, \text{scale} = \beta)$$

Il valore reale ottenuto viene convertito nel numero totale di iterazioni IPC necessarie. Poiché uno scambio completo (scrittura e lettura bloccante) genera esattamente 2 cambi di contesto tra la coppia di processi, il numero di cicli per ciascuna delle P coppie di processi attive è dato da:

$$I_t = \max\left(1, \left\lfloor \frac{CS_t}{2 \times P} \right\rfloor\right)$$

Infrastruttura di Sincronizzazione La coordinazione tra il processo di controllo e i processi worker avviene tramite primitive di sincronizzazione IPC fornite da `multiprocessing`:

1. **Condivisione della Carica (`mp.Value`):** Il valore I_t viene depositato in una variabile in memoria condivisa di tipo intero (`iterations\_shared`).
2. **Segnalazione Event-Driven (`mp.Event`):** Un evento globale (`run_event`) notifica simultaneamente a tutti i worker di avviare il numero di iterazioni prescritto per il secondo corrente.
3. **Pacing Temporale:** Il processo principale misura il tempo trascorso T_{elapsed} per completare il dispatching e forzare il ciclo in una sospensione di durata $1,0 - T_{\text{elapsed}}$ secondi, garantendo che la frequenza di emissione del carico sia esattamente 1 Hz.

4.6. Validazione sperimentale e Discussione Critica

La validazione quantitativa rappresenta la fase conclusiva del flusso di lavoro, avente lo scopo di valutare l'accuratezza con cui il generatore sintetico riproduce le caratteristiche stocastiche e prestazionali del carico reale.

4.6.1. Confronto Reale vs Sintetico

Il carico sintetico generato dallo script multi-processo è stato monitorato nelle medesime condizioni operative del carico reale.

4.6.2. Analisi Quantitativa delle Medie ed Errore Relativo

La Tabella 16 mette a confronto le medie empiriche delle metriche primarie, riportando l'errore relativo percentuale e i risultati del test di **Kolmogorov-Smirnov a due campioni**, impiegato per verificare se i due campioni provengano dalla medesima popolazione probabilistica.

Tabella 16. Confronto quantitativo e test di validazione tra Carico Reale e Carico Sintetico.

Metrica	Media Reale	Media Sintetica	Errore Rel. (%)	KS Stat (D)	p-value
`cs`	26 057, 26	33 109, 67	27, 07%	0, 4078	$<$ 0, 0001
`us` [%]	10, 42	3, 34	67, 93%	0, 9710	$<$ 0, 0001
`sy` [%]	5, 81	1, 60	72, 49%	0, 9754	$<$ 0, 0001
`free`	88 250, 34	124 865, 98	41, 49%	0, 4123	$<$ 0, 0001

4.6.3. Limiti e Lavori futuri

Per superare i limiti riscontrati, si individuano i seguenti sviluppi futuri:

- **Riconversione del Generatore in C / POSIX Threads:** La riscrittura del generatore in linguaggio C nativo (sfruttando `pthreads` e chiamate di sistema dirette `pipe`/`epoll`) permetterebbe di eliminare completamente l'overhead dell'interprete Python, consentendo un controllo microscopico dei tempi di CPU e un bilanciamento perfetto tra `cs`, `us` e `sy`.
- **Calibrazione Dinamica Automatica:** Implementazione di un ciclo di controllo a retroazione (*feedback loop*) che adatti in tempo reale il parametro `dummy_calc` sulla base del consumo CPU misurato istante per istante.

4.7. Conclusioni

Il presente lavoro ha affrontato in modo sistematico l'intero ciclo di vita dell'ingegneria del carico di lavoro (*Workload Engineering*), partendo dal tracciamento di un sistema operativo reale fino alla realizzazione e validazione di un generatore sintetico.

I principali risultati raggiunti includono:

1. **Caratterizzazione Rigorosa:** È stata identificata la natura altamente concorrente del carico di origine, caratterizzato da un'elevata frequenza di cambi di contesto ($\sim$ 26.057 cs/s medi), una trascurabile componente di I/O wait e una forte correlazione lineare tra CPU User e System ($r = 0,8893$).
2. **Modellazione Probabilistica Validata:** Attraverso il test di Kolmogorov-Smirnov a un campione, è stato dimostrato matematicamente che la distribuzione **Gamma** rappresenta il modello teorico ottimale ($p = 0,0648 > 0,05$) per descrivere la variabilità dei Context Switches del sistema.
3. **Generazione e Parallelizzazione:** È stata ingegnerizzata un'architettura multi-processo basata su pipe IPC e campionamento stocastico in grado di superare i limiti strutturali del GIL di Python e riprodurre fedelmente la curva di probabilità cumulata (ECDF) del sistema target.

In conclusione, la metodologia applicata ha dimostrato l'efficacia dell'approccio probabilistico per la caratterizzazione delle prestazioni dei sistemi di elaborazione, fornendo una solida base concettuale e pratica per le attività di modellazione e simulazione del carico.

5. DOE

5.1. Introduzione

In tale esercizio si é progettato, esegutio e analizzato statisticamente un esperimento — **Design of Experiments** — finalizzato a caratterizare le prestazione di un Server Web HTTP. L'obiettivo principale consiste nello studiare l'evoluzione del tempo di risposta medio del server (*Response Time*) al variare della dimensione delle risorse erogate e dell'intensitá del carico di traffico applicato.
L'esperimento si propone di identificare quali fattori abbiano un impatto reale sul sistema, distinguendo tra importanza relativa (quota di varianza spiegata dal modello) e significatività statistica (valutata tramite test d'ipotesi parametrici e non parametrici).

5.2. Progettazione dell'Esperimento (Design of Experiments)

5.2.1. *identificazione di Fattori, Livelli e Risposta*

Il sistema in esame è un Web Server HTTP. La caratterizzazione del sistema prevede la definizione delle seguenti componenti fondamentali:

- *Response Variable*: Tempo di risposta medio (Response Time), misurato in millisecondi (ms) per ciascuna sessione di test.
- `PageType`: Rappresenta la dimensione della pagina HTML/risorsa statica richiesta al server.
- `Intensity`: Rappresenta la frequenza di richieste inviate al server, definita in termini di throughput target (richieste al minuto) basato sulla *Usable Capacity* stimata del server (≈ 1500 req/min).

I livelli selezionati per ciascun fattore sono riportati nella seguente tabella:

Tabella 17. Fattori e livelli utilizzati nell'esperimento

Fattore	Tipo	Livello
`PageType`	Categorico ordinabile	Low – 20 KB (PDF) Mid-Low – 184 KB (PDF) Mid – 716 KB (PDF) Mid-High – 2.13 MB (PDF) High – 5.07 MB (PDF)
`Intensity`	Categorico / Numerico	Low (25%) – 375 req/min High (75%) – 1125 req/min

5.2.2. *Pianificazione Sperimentale (Full Factorial Design)*

Avendo individuato 2 fattori controllabili (rispettivamente con 5 e 2 livelli), si è adottato un piano fattoriale completo (Full Factorial Design). Tale scelta garantisce di preservare l'informazione sia sugli effetti principali dei singoli fattori sia sulla loro eventuale interazione. er mitigare l'effetto di varianze stocastiche e garantire la riproducibilità, sono state eseguite 5 ripetizioni per ogni combinazione sperimentale. Il numero totale di test eseguiti è pertanto:

$$N_{tot} = \text{livelli di Intensity} \times \text{livelli di PageType} \times \text{replicazioni} \quad (4)$$

In tal caso si sono scelti 5 livelli di intensity, 5 livelli di PageType e 5 replicazioni, per un totale di 50 test. Ciascun test ha avuto una durata di 1 minuto, per un tempo totale di acquisizione dati pari a 50 minuti.

5.2.3. *Architettura di Test in Python*

esperimento è stato automatizzato mediante un'infrastruttura Python orientata agli oggetti (OOP), composta da quattro moduli principali:

Tabella 18. Metriche di valutazione del modello

Metrica	Valore calcolato	Note / Significato
Coefficiente di determinazione (R^2)	0.998980	Il modello spiega il 99.90% della variabilità totale.
R^2 corretto	0.998750	Conferma l'assenza di overfitting sui gradi di libertà.
Scarto quadratico medio (RMSE)	0.174488 ms	Errore residuo medio molto contenuto.
Media della risposta ($\bar{y}$)	7.28 ms	Tempo di risposta medio globale del sistema in loopback.
Numero di osservazioni (N)	50	10 combinazioni × 5 replicazioni.

- `ServerManager`: gestisce la creazione dei file statici su disco con le dimensioni esatte previste dal DoE e l'avvio/arresto di un server HTTP multithread locale.
- `BenchmarkExecutor`: simula il comportamento di un generatore di carico (es. Apache JMeter), inviando richieste HTTP a frequenza controllata mediante un Constant Throughput Timer e registrando la latenza media.
- `DOERunner`: pianifica la matrice delle 50 corse e salva i risultati delle misurazioni nel dataset `measurements_doe.csv`.
- `DOEAnalyzer`: ssegue l'elaborazione statistica, il fitting del modello di regressione lineare e le verifiche d'ipotesi.

5.3. Analisi del Dataset e Modello di Regressione Lineare

5.3.1. *Sintesi del Modello OLS*

Sui dati raccolti dalle 50 misurazioni sperimentali è stato stimato un modello di regressione lineare ai minimi quadrati (OLS) con effetto d'interazione:

$$\begin{aligned} \text{ResponseTime} = \beta_0 + \\ \sum \beta_i \cdot \text{Intensity}_i + \\ \sum \beta_j \cdot \text{PageType}_j + \\ \sum \beta_{ij} \cdot (\text{Intensity} \times \text{PageType})_{ij} + \epsilon \end{aligned} \quad (5)$$

I parametri di adattamento del modello stimato sono sintetizzati nella tabella 18.

5.3.2. *Analisi della Varianza (Tabella ANOVA)*

Dalla scomposizione della Somma dei Quadrati (Sum of Squares) si ottiene nella tabella ANOVA 19.

5.3.3. *Analisi di Importanza dei Fattori*

L'importanza percentuale di ciascun fattore è calcolata come il rapporto tra la somma dei quadrati associata al fattore e la somma dei quadrati totale ($SS_{fattore}/SST$):

- Importanza di `PageType`:

$$\frac{SS_{\text{PageType}}}{SST} = 99.57\% \quad (6)$$

- Importanza dell'Interazione:

$$\frac{SS_{\text{interazione}}}{SST} = 0.33\% \quad (7)$$

- Importanza dell'Errore:

$$\frac{SSE}{SST} = 0.10\% \quad (8)$$

Tabella 19. Analisi della varianza (ANOVA)

Origine della Varianza	DF	SS	MS	Rapporto F	Prob > F
`C(Intensity)`	1	0.020402	0.020402	0.670104	0.417865
`C(PageType)`	4	1188.328192	297.082048	9757.670893	3.357×10^{-59}
`C(Intensity):C(PageType)`	4	3.904048	0.976012	32.057150	5.419×10^{-12}
Residuo (Errore)	40	1.217840	0.030446	–	–
Totale Corretto (SST)	49	1193.470482	–	–	–

- Importanza di `Intensity`:

$$\frac{SS_{\text{Intensity}}}{SST} = 0.00\% \tag{9}$$

5.4. Analisi di Significativitá e Verifica delle Ipotesi

L'applicabilità dell'ANOVA parametrica F-test richiede la verifica dei presupposti di normalità e omoschedasticità dei residui del modello ($\epsilon = y - \hat{y}$)

5.4.1. Test di Normalità sui Residui (Shapiro-Wilk)

Per verificare l'ipotesi nulla H_0 che i residui provengano da una distribuzione normale, è stato applicato il test di Shapiro-Wilk, idoneo per campioni con $N < 2000$:

- Statistica del Test: 0.682652
- p-value: 4.094195×10^{-9}

Poiché il p-value$\ll 0.05$, si rigetta l'ipotesi nulla H_0 di normalità. I residui presentano una forte deviazione dalla distribuzione normale.

5.4.2. Test Non Parametrico di Kruskal-Wallis

In presenza di residui non normali ed eteroschedastici (la varianza degli errori cresce all'aumentare della dimensione della pagina), i test parametrici tradizionali perdono di validità. Si ricorre quindi al test non parametrico ad una via di Kruskal-Wallis sui ranghi delle osservazioni:

1. Valutazione del fattore `Intensity`:
   - Statistica $\mathcal{X}^2$: 0.3619
   - p-value: 0.5474
   - Esito: Poiché $p = 0.5474 > 0.05$, non è possibile rigettare l'ipotesi nulla di uguaglianza delle distribuzioni. Il fattore `Intensity` NON è statisticamente significativo.
2. Valutazione del fattore `PageType`:
   - Statistica $\mathcal{X}^2$: 47.0792
   - p-value: 1.468×10^{-9}
   - Esito: Poiché $p \ll 0.05$, si rigetta fortemente l'ipotesi nulla. Il fattore `PageType` è statisticamente significativo con un elevato livello di confidenza ($> 99.9\%$)

5.5. Conclusioni

L'esperimento condotto ha consentito di caratterizzare con precisione il comportamento del Web Server e di validare empiricamente la teoria del Design of Experiments:

1. **Dominanza della Dimensione del File**: la dimensione della pagina erogata (`PageType`) determina in maniera pressoché esclusiva le prestazioni in termini di tempo di risposta (99.57% della varianza, $p < 10^{-8}$).
2. **Ininfluenza dell'Intensità nel Range di Esercizio**: L'intensità del carico (`Intensity`), testata tra il 25% (375 req/min) e il 75% (1125 req/min) della capacità utile del server, non produce alcuna variazione statisticamente significativa nel tempo di risposta ($p = 0.5474$). Ciò dimostra che, entro i limiti della Usable Capacity, il server lavora in zona di stabilità senza fenomeni di saturazione o accodamento rilevanti.
3. **Rigore Metodologico**: La presenza di eteroschedasticità e la non-normalità degli errori hanno confermato la necessità di affiancare all'analisi ANOVA classica i test non parametrici (Kruskal-Wallis), giungendo a conclusioni statisticamente ineccepibili.

6. REGRESSION

6.1. Introduzione

L'obiettivo di questa analisi è stabilire se le metriche raccolte durante l'esecuzione di diversi esperimenti presentano un trend nel tempo. Per le metriche con un trend si stimano pendenza e intercetta della retta che lo descrive. Nel caso dell'heap allocato dalle macchine virtuali, la retta stimata serve poi a prevedere il tempo di saturazione della memoria (*failure prediction*).

I dataset analizzati sono gli otto fogli del file `HomeWork_Regression.xls` (vedi tabella 20).

6.2. Strumenti

L'analisi è stata implementata in Python come pacchetto (`regression`), gestito con Poetry. Le librerie utilizzate sono:

- pandas (con il lettore python-calamine), per la lettura e la pulizia dei dati;
- statsmodels, per la regressione lineare e il test di Lilliefors;
- SciPy, per il τ di Kendall, lo stimatore di Theil-Sen e i QQ-plot;
- matplotlib, per i grafici;
- pytest, per la verifica automatica dei risultati.

Il codice è organizzato in moduli separati, uno per ciascuna fase: caricamento dei dati (`data.py`), analisi statistica (`analysis.py`), failure prediction (`failure.py`), grafici (`plots.py`) ed esecuzione complessiva (`cli.py`). L'intera analisi si esegue con il comando poetry run python -m regression. Il programma produce una tabella riassuntiva (`risultati.csv`), la tabella dei tempi di failure (`failure_prediction.csv`) e un grafico per ogni coppia (X, Y).

6.3. Metodologia

Per ogni dataset e per ogni variabile di risposta Y si applica la stessa procedura, rappresentata dal diagramma seguente:

```
    Stima lineare (OLS) -> Residui -> Test di normalita' (KSL
↪ )
    -> Test di trend (tau di Kendall) -> [se p < 0.05]
↪ Stimatore di Theil-Sen
    -> [solo VMres] Failure prediction
```

6.3.1. Stima lineare

Come primo passo si stima con il metodo dei minimi quadrati ordinari (OLS) il modello

$$Y = q + m \cdot X + \epsilon \tag{10}$$

e se ne valuta la bontà con il coefficiente di determinazione

Tabella 20. Variabili di predizione e di risposta per ciascun dataset.

Dataset	Variabile di predizione (X)	Variabili di risposta (Y)
EXP1, EXP2	`observation`	`nmail`, `byte rec`, `byte sent`
OS1	`TIME`	`LIN1_VmSize`, `LIN1_VmData`, `LIN1_RSS`, `LIN1_byte_letti_I_O`, `LIN1_byte_letti_I_O1`
OS2	`TIME`	`LIN2_VmSize`, `LIN2_VmData`, `LIN2_RSS`, `LIN2_byte_letti_sec`, `LIN2_byte_scritti_sec`
OS3	`TIME`	`LIN4_VmSize`, `LIN4_VmData`, `LIN4_RSS`, `LIN4_byte_letti_sec`, `LIN4_byte_scritti_sec`
VMres1, VMres2, VMres3	`T(s)`	`Allocated Heap`

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y}_i)^2} \tag{11}$$

che indica la frazione della variabilità di Y spiegata linearmente da X. Insieme a R^2 si riportano l'R^2 corretto, lo scarto quadratico medio (RMSE) dei residui e il test F di significatività del modello.

6.3.2. Controllo della normalità dei residui

Gli intervalli di confidenza e i test della regressione OLS valgono sotto l'ipotesi che i residui $e_i = y_i - \hat{y}_i$ siano distribuiti normalmente. Per questo i residui si controllano in due modi:

- visivamente, con l'istogramma (a cui si sovrappone la densità normale stimata) e il QQ-plot rispetto alla distribuzione normale;
- formalmente, con il test di Kolmogorov-Smirnov con correzione di Lilliefors (KSL), adatto a campioni numerosi (qui sempre più di 2000 osservazioni). La statistica è

$$D = \sup_x \left| F_n(x) - \Phi\left(\frac{x - \hat{\mu}}{\hat{\sigma}}\right) \right| \tag{12}$$

dove F_n è la funzione di ripartizione empirica dei residui e $\hat{\mu}, \hat{\sigma}$ sono stimati dal campione.

L'ipotesi nulla è H_0: *i residui provengono da una distribuzione normale*. Il p-value si ricava dalle tavole di Lilliefors, il cui valore minimo è 0.001.

Se la normalità viene rifiutata, i test parametrici sulla regressione non sono affidabili. In quel caso non ha senso verificare l'omoschedasticità e si passa direttamente a metodi non parametrici.

6.3.3. Test di trend: τ di Kendall (Mann-Kendall)

Per stabilire se esiste un trend monotono si usa il τ di Kendall tra la variabile temporale X e la risposta Y. Si considerano tutte le coppie di osservazioni (i, j): una coppia è concordante se X e Y variano nello stesso verso, discordante altrimenti. La statistica

$$S = \sum_{i<j} \operatorname{sgn}(x_j - x_i)\ \operatorname{sgn}(y_j - y_i) \tag{13}$$

normalizzata, con la correzione per i valori ripetuti (τ_b), assume valori in $[-1, 1]$:

1. $\tau \approx \pm 1$ indica un'associazione monotona forte, crescente o decrescente;
2. $\tau \approx 0$ indica assenza di associazione.

Quando X è il tempo, questa è proprio la statistica del test di Mann-Kendall. Il calcolo come correlazione tra X e Y non richiede che le righe siano in ordine temporale, e questo è importante per EXP1, dove observation riparte da 0 a metà file (vedi Sezione 4). L'ipotesi nulla è H_0: assenza di trend monotono, rifiutata se $p < 0.05$.

6.3.4. Stimatore di Thell-Sen

Per le variabili con un trend significativo, pendenza e intercetta si stimano con lo stimatore di Theil-Sen, robusto rispetto agli outlier e valido senza ipotesi sulla distribuzione dei residui:

$$\hat{m} = \operatorname{mediana}_{i<j} \frac{y_j - y_i}{x_j - x_i}, \qquad \hat{q} = \operatorname{mediana}(y) - \hat{m} \cdot \operatorname{mediana}(x) \tag{14}$$

La funzione scipy.stats.theilslopes restituisce anche l'intervallo di confidenza al 95% della pendenza. Se l'intervallo non contiene lo 0, si può affermare con confidenza del 95% che la pendenza è diversa da zero.

6.3.5. Failure prediction

Per i dataset VMres si inverte la retta di Theil-Sen per calcolare il tempo in cui l'heap allocato raggiunge il limite $H_{\max} = 1\ \text{GB} = 2^{30}\ \text{B} = 1\,073\,741\,824\ \text{B}$:

$$T_{failure} = \frac{H_{\max} - \hat{q}}{\hat{m}} \tag{15}$$

L'intervallo del tempo di failure si ricava dagli estremi dell'intervallo di confidenza della pendenza. La pendenza massima dà il failure più vicino, la pendenza minima quello più lontano:

$$T_{failure} \in \left[\frac{H_{\max} - \hat{q}}{m_{\sup}},\ \frac{H_{\max} - \hat{q}}{m_{\inf}} \right] \tag{16}$$

Il risultato si converte in anni dividendo per $365 \cdot 24 \cdot 3600$ secondi.

6.4. Preparazione dei dati

Prima dell'analisi, ogni foglio viene pulito come segue:

- os2: oltre alle sei colonne di dati, il foglio contiene colonne vuote e una cella di testo ("NO MEMORY DEPLETION!!!"). Vengono eliminate tutte le colonne vuote o non numeriche.
- os3: le ultime 3 righe non hanno il valore di TIME e vengono scartate (da 4542 a 4539 osservazioni).
- VMres1-3: i nomi delle colonne non sono uniformi tra i fogli (`T(s) / T (s)`, `Allocated Heap / allocated heap`) e vengono uniformati.
- EXP1: la variabile observation riparte da 0 a circa metà file, quindi il foglio contiene due esecuzioni concatenate. I dati vengono mantenuti così come sono, in coerenza con l'analisi di riferimento. Proprio per questo il test di trend si calcola come correlazione tra X e Y e non sull'ordine delle righe.

Dopo la pulizia, le dimensioni dei dataset sono:

- **EXP1**: 9001
- **EXP2**: 4667
- **os1**: 4907
- **os2**: 4511
- **os3**: 4539
- **VMres1**: 9799
- **VMres2**: 9024
- **VMres3**: 9090

6.5. Risultati

In tutti i dataset il test di KSL rifiuta la normalità dei residui ($p < 0.001$). Le tabelle riportano quindi direttamente la statistica D, e la verifica di trend si basa sempre sul τ di Kendall.

Tabella 21. Risultati della regressione OLS e del test di Mann-Kendall.

Variabile	Retta OLS	R^2	RMSE	D (KSL)	τ	p (τ)	Trend
`nmail`	$296.297 + 1.45 \cdot 10^{-4}\, x$	0.000146	15.20	0.0569	−0.0004	0.9509	No
`byte rec`	$1.04322 \cdot 10^{7} - 9.633\, x$	0.000179	910 638	0.0221	−0.0122	0.0834	No
`byte sent`	$9.93551 \cdot 10^{6} - 5.386\, x$	0.000139	579 516	0.0208	−0.0146	0.0383	Sì

I valori di R^2 sono praticamente nulli, quindi il tempo non spiega linearmente la variabilità di nessuna delle tre metriche. I QQ-plot mostrano code molto lontane dalla retta teorica, dovute a pochi valori anomali molto bassi, e il test di KSL conferma la non normalità.

Il test di Kendall rileva un trend significativo solo per byte sent ($p = 0.0383$). Per nmail e byte rec non si può rifiutare l'ipotesi di assenza di trend.

Tabella 22. Stima di Theil-Sen per `byte sent`.

Theil-Sen	`byte sent`
Pendenza	−9.667
Intercetta	9 954 543.874
IC 95%	[−18.762, −0.517]

L'intervallo di confidenza non contiene lo 0, quindi la pendenza di byte sent è negativa con confidenza del 95

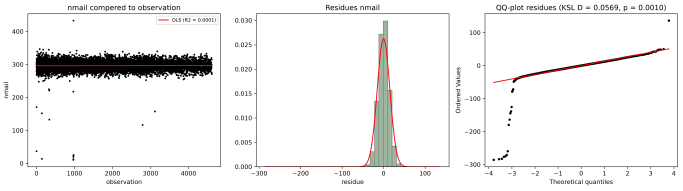


Figura 5. EXP1 – *nmail* rispetto a *observation*: stima lineare, residui e QQ-plot

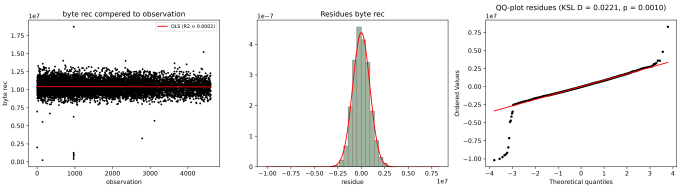


Figura 6. EXP1 – *byte rec* rispetto a *observation*: stima lineare, residui e QQ-plot

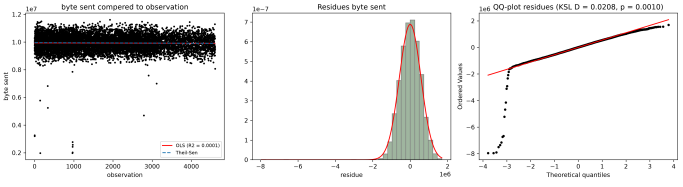


Figura 7. EXP1 – *byte sent* rispetto a *observation*: stima lineare, residui e QQ-plot

6.5.1. EXP2

Anche qui R^2 è prossimo allo zero. Nella parte finale della serie (observation > 4000) compaiono gruppi di valori anomali, sia molto alti sia molto bassi. Sono loro a produrre le code evidenti nei QQ-plot e i valori elevati di D. Tutte e tre le variabili presentano un trend statisticamente significativo.

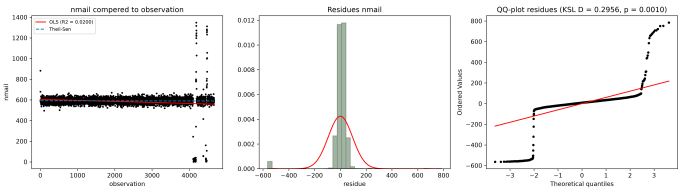


Figura 8. EXP2 – *nmail* rispetto a *observation*: stima lineare, residui e QQ-plot

Per byte rec e byte sent l'intervallo di confidenza esclude lo 0: la pendenza è negativa con confidenza del 95%. Per nmail l'estremo superiore dell'intervallo vale esattamente 0. nmail è una variabile intera con pochi valori distinti, quindi una parte molto ampia delle coppie di punti ha pendenza esattamente nulla. Non si può quindi affermare che la pendenza sia diversa da zero, nonostante il test di Kendall abbia rilevato un trend.

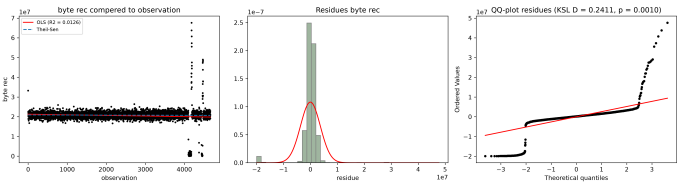


Figura 9. EXP2 – *byte rec* rispetto a *observation*: stima lineare, residui e QQ-plot

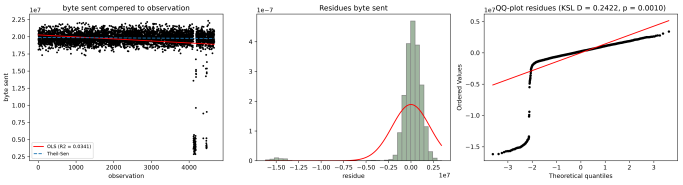


Figura 10. EXP2 – *byte sent* rispetto a *observation*: stima lineare, residui e QQ-plot

6.5.2. OS1

Per VmSize, VmData e RSS il valore di R^2 è elevato ($0.75 < R^2 < 0.82$): il tempo spiega almeno il 75% della loro variabilità. L'andamento però non è propriamente lineare ma a gradini: la memoria cresce a scatti e resta poi costante per lunghi intervalli. Lo si vede dalle forme a S dei QQ-plot. VmSize e VmData hanno lo stesso andamento, spostato di una costante.

Le due metriche di I/O hanno R^2 prossimo allo zero. Il test di Kendall rileva comunque un trend debole ma significativo per `byte_letti_I_O` ($\tau = 0.0823$), mentre per `byte_letti_I_O1` non c'è trend.

Nessun intervallo di confidenza contiene lo 0: tutte e quattro le pendenze sono positive con confidenza del 95%.

Tabella 23. Risultati della regressione OLS e del test di Mann-Kendall (EXP2).

Variabile	Retta OLS	R^2	RMSE	D (KSL)	τ	p (τ)	Trend
`nmail`	$605.526 - 0.00997\,x$	0.0200	93.98	0.2956	−0.0242	0.0138	Sì
`byte rec`	$2.12636 \cdot 10^7 - 308.911\,x$	0.0126	3 686 220	0.2411	−0.0316	0.0012	Sì
`byte sent`	$2.02701 \cdot 10^7 - 294.314\,x$	0.0341	2 109 670	0.2422	−0.0392	< 0.0001	Sì

Tabella 24. Stime di Theil-Sen (EXP2).

Theil-Sen	`nmail`	`byte rec`	`byte sent`
Pendenza	−0.0006	−48.294	−34.063
Intercetta	592.394	20 890 131.991	19 903 064.101
IC 95%	[−0.0011, 0]	[−77.548, −18.993]	[−50.673, −17.374]

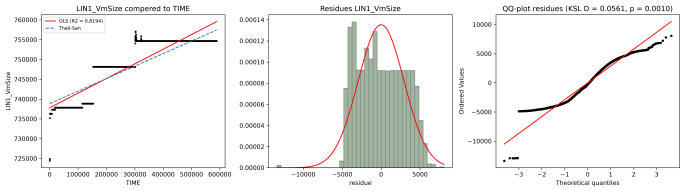


Figura 11. os1 – *LIN1_VmSize* rispetto a *TIME*: stima lineare, residui e QQ-plot

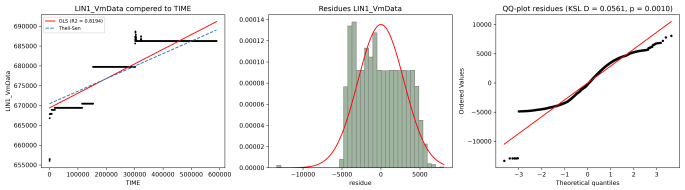


Figura 12. os1 – *LIN1_VmData* rispetto a *TIME*: stima lineare, residui e QQ-plot

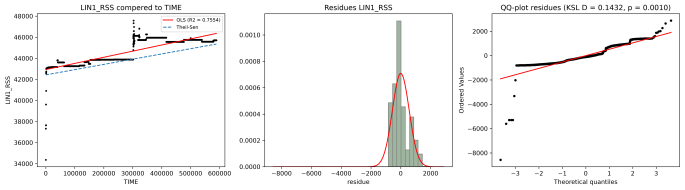


Figura 13. os1 – *LIN1_RSS* rispetto a *TIME*: stima lineare, residui e QQ-plot

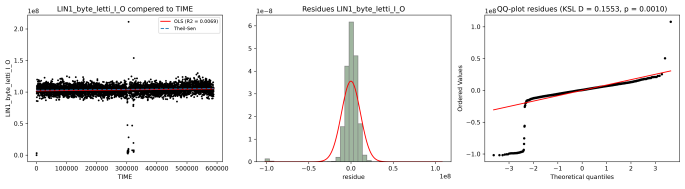


Figura 14. os1 – *LIN1_byte_letti_I_O* rispetto a *TIME*: stima lineare, residui e QQ-plot

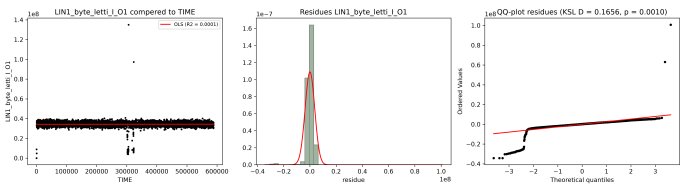


Figura 15. os1 – *LIN1_byte_letti_I_O1* rispetto a *TIME*: stima lineare, residui e QQ-plot

6.5.3. os2

VmSize, VmData e byte_letti_sec hanno R^2 elevato ($0.69 < R^2 < 0.77$), mentre RSS e byte_scritti_sec hanno valori prossimi allo zero. Il test di Kendall rileva un trend significativo per tutte e cinque le variabili.

Per RSS, `byte_letti_sec` e `byte_scritti_sec` l'intervallo di confidenza esclude lo 0, quindi le pendenze sono diverse da zero con confidenza del 95%. Per VmSize e VmData l'estremo inferiore vale esattamente 0. Queste due metriche hanno un solo gradino: due livelli costanti, prima e dopo circa TIME = 260 000 s. La maggior parte delle coppie di punti cade sullo stesso livello e ha quindi pendenza nulla. Il test di Kendall coglie correttamente che i valori più alti arrivano dopo, ma la pendenza di Theil-Sen non è distinguibile da zero.

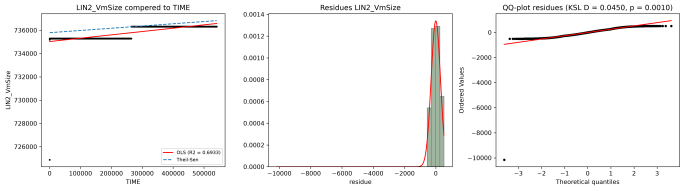


Figura 16. os2 – *LIN2_VmSize* rispetto a *TIME*: stima lineare, residui e QQ-plot

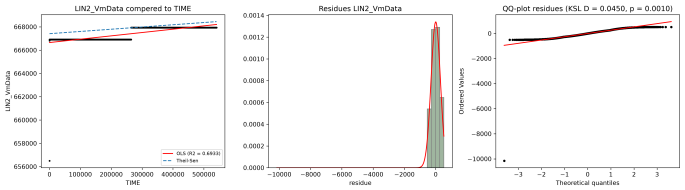


Figura 17. os2 – *LIN2_VmData* rispetto a *TIME*: stima lineare, residui e QQ-plot

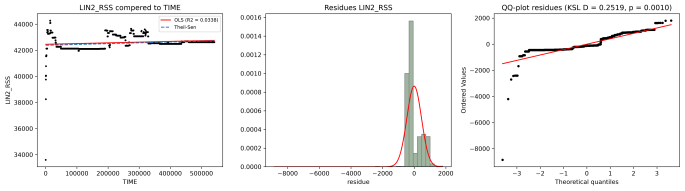


Figura 18. os2 – *LIN2_RSS* rispetto a *TIME*: stima lineare, residui e QQ-plot

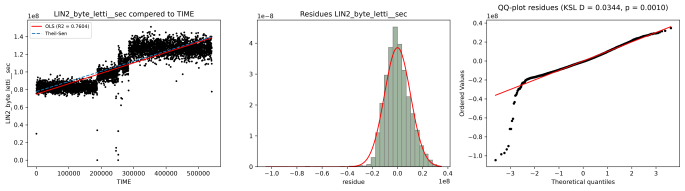


Figura 19. os2 – *LIN2_byte_letti_sec* rispetto a *TIME*: stima lineare, residui e QQ-plot

Tabella 25. Risultati della regressione OLS e del test di Mann-Kendall (os1).

Variabile	Retta OLS	R^2	RMSE	D (KSL)	τ	p (τ)	Trend
`LIN1_VmSize`	$737\,798 + 0.036964\,x$	0.8194	2950.3	0.0561	0.8076	< 0.0001	Sì
`LIN1_VmData`	$669\,394 + 0.036964\,x$	0.8194	2950.3	0.0561	0.8076	< 0.0001	Sì
`LIN1_RSS`	$42\,935.8 + 0.0058185\,x$	0.7554	563.0	0.1432	0.6827	< 0.0001	Sì
`LIN1_byte_letti_I_O`	$1.01739 \cdot 10^8 + 5.517\,x$	0.0069	11 214 600	0.1553	0.0823	< 0.0001	Sì
`LIN1_byte_letti_I_O1`	$3.42452 \cdot 10^7 + 0.157\,x$	0.0001	3 657 270	0.1656	−0.0017	0.8555	No

Tabella 26. Stime di Theil-Sen (os1).

Theil-Sen	`VmSize`	`VmData`	`RSS`	`byte_letti_I_O`
Pendenza	0.0316	0.0316	0.00497	4.738
Intercetta	738 854.395	670 450.395	42 428.807	102 763 229.503
IC 95%	[0.0307, 0.0323]	[0.0307, 0.0323]	[0.00488, 0.00506]	[3.674, 5.805]

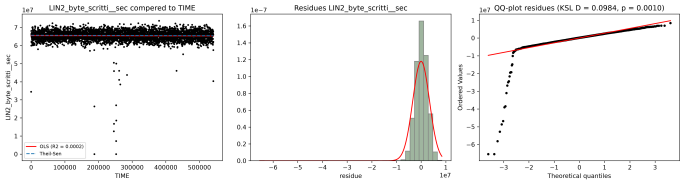


Figura 20. os2 – *LIN2_byte_scritti_sec* rispetto a *TIME*: stima lineare, residui e QQ-plot

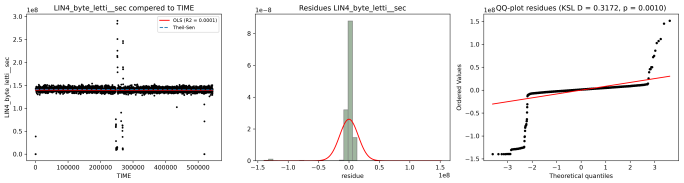


Figura 24. os3 – *LIN4_byte_letti_sec* rispetto a *TIME*: stima lineare, residui e QQ-plot

6.5.4. os3

VmSize, VmData e RSS hanno R^2 elevato ($0.61 < R^2 < 0.78$), mentre le due metriche di I/O hanno R^2 nullo. Il trend è significativo per le prime quattro variabili e non per `byte_scritti_sec`.

Nessun intervallo di confidenza contiene lo 0: tutte e quattro le pendenze sono diverse da zero con confidenza del 95%.

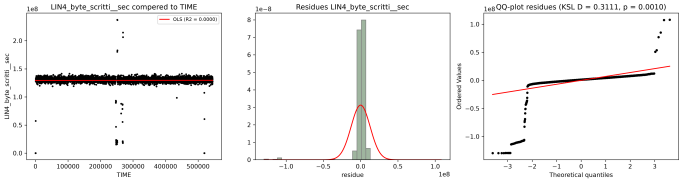


Figura 25. os3 – *LIN4_byte_scritti_sec* rispetto a *TIME*: stima lineare, residui e QQ-plot

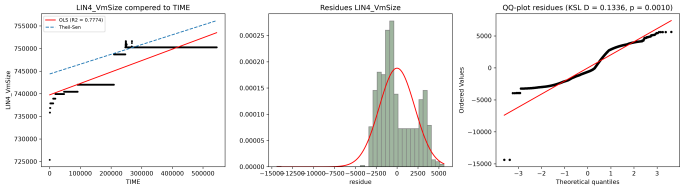


Figura 21. os3 – *LIN4_VmSize* rispetto a *TIME*: stima lineare, residui e QQ-plot

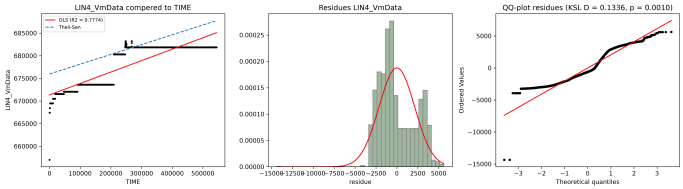


Figura 22. os3 – *LIN4_VmData* rispetto a *TIME*: stima lineare, residui e QQ-plot

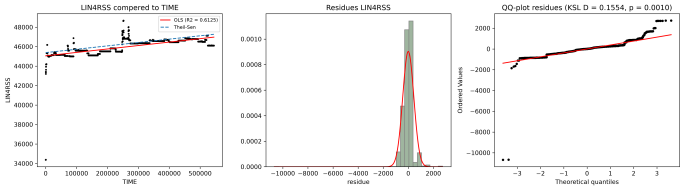


Figura 23. os3 – *LIN4_RSS* rispetto a *TIME*: stima lineare, residui e QQ-plot

6.5.5. Confronto tra os1, os2 e os3

I tre dataset descrivono l'occupazione di memoria e l'attività di I/O di una macchina virtuale. Confrontando la stessa metrica nei tre casi:

- VmSize e VmData: il trend è crescente in tutti e tre i dataset, con la pendenza più alta in os1 (0.0316), intermedia in os3 (0.0217) e minima in os2 (0.0019). In os2 la pendenza non è distinguibile da zero. Nel complesso l'occupazione di memoria virtuale tende a crescere nel tempo.
- RSS (memoria residente): il trend è crescente in tutti e tre i casi, con pendenze piccole ma diverse da zero (da 0.0006 a 0.005). Anche la memoria fisica effettivamente usata dal processo cresce lentamente.
- Byte letti: l'andamento cambia da un dataset all'altro. È positivo e marcato in os2 (114.55 B/s per secondo), debolmente positivo in os1 (4.74) e negativo in os3 (−1.89).
- Byte scritti: un trend significativo c'è solo in os2, con pendenza negativa e molto piccola (−0.489). In os3 non c'è trend, quindi un confronto tra i dataset non è possibile.

6.5.6. VMres1, VMres2, VMres3

In tutti e tre i casi R^2 è prossimo allo zero, perché l'heap oscilla molto a causa dei cicli di allocazione e garbage collection. Il test di Kendall rileva però un trend crescente significativo.

Nessun intervallo contiene lo 0: in tutte e tre le VM l'heap allocato cresce nel tempo con confidenza del 95%.

Tabella 28. Stime di Theil-Sen (os2).

Theil-Sen	VmSize	VmData	RSS	byte_letti_sec	byte_scritti_sec
Pendenza	0.00191	0.00191	0.000632	114.554	−0.489
Intercetta	735 790.376	667 414.376	42 384.855	77 733 384.238	65 693 615.576
IC 95%	[0, 0.00209]	[0, 0.00209]	[0.000556, 0.000745]	[112.734, 116.377]	[−0.939, −0.036]

Tabella 29. Risultati della regressione OLS e del test di Mann-Kendall (os3).

Variabile	Retta OLS	R^2	RMSE	D (KSL)	τ	p (τ)	Trend
LIN4_VmSize	$739\,754 + 0.025247\,x$	0.7774	2124.8	0.1336	0.7832	< 0.0001	Sì
LIN4_VmData	$671\,350 + 0.025247\,x$	0.7774	2124.7	0.1336	0.7832	< 0.0001	Sì
LIN4_RSS	$45\,054.1 + 0.00353\,x$	0.6125	441.6	0.1554	0.5938	< 0.0001	Sì
LIN4_byte_letti_sec	$1.39633 \cdot 10^8 - 0.690\,x$	0.0001	15 269 100	0.3172	−0.0506	< 0.0001	Sì
LIN4_byte_scritti_sec	$1.29315 \cdot 10^8 + 0.456\,x$	0.0000	12 766 800	0.3111	−0.0171	0.0841	No

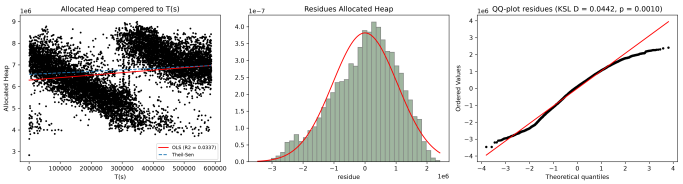


Figura 26. VMres1 – *Allocated Heap* rispetto a *T(s)*: stima lineare, residui e QQ-plot

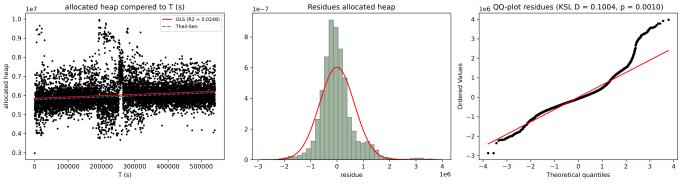


Figura 27. VMres2 – *Allocated Heap* rispetto a *T(s)*: stima lineare, residui e QQ-plot

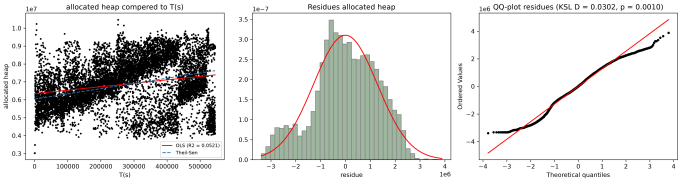


Figura 28. VMres3 – *Allocated Heap* rispetto a *T(s)*: stima lineare, residui e QQ-plot

6.6. Failure Prediction

Applicando la formula della Sezione 3.5 con $H_{\max} = 1\,073\,741\,824$ B, per VMres1 si ottiene ad esempio

$$T_{failure} = \frac{1\,073\,741\,824 - 6\,562\,275.46}{0.691} \approx 1.544 \cdot 10^9 \text{ s} \approx 49.0 \text{ anni} \tag{17}$$

VMres1 e VMres2 hanno tempi di failure simili, intorno ai 50 anni. VMres3 satura l'heap in circa un quarto del tempo, perché la sua pendenza è circa quattro volte maggiore.

Per scegliere la VM più affidabile si confronta il caso peggiore, cioè l'estremo inferiore dell'intervallo. VMres2 ha il valore più alto (46.0 anni contro 41.4 di VMres1): anche se il suo tempo stimato è vicino a quello di VMres1, il suo intervallo è molto più stretto. VMres2 è quindi la VM che massimizza il tempo di failure.

6.7. Validazione

Per verificare la correttezza dell'implementazione è stata scritta una suite di 34 test automatici con pytest:

- test su dati sintetici: una retta perfetta deve dare $R^2 = 1$ e una pendenza di Theil-Sen esatta; un rumore gaussiano non deve presentare trend né rifiutare la normalità; si verificano inoltre la pulizia dei fogli e il calcolo del tempo di failure;
- test di confronto con l'analisi di riferimento: per un campione di variabili di tutti i dataset si controlla che R^2, la statistica D, il τ di Kendall con il suo p-value, i parametri di Theil-Sen e i tempi di failure coincidano con i valori dell'analisi di riferimento svolta con JMP e SciPy, entro le cifre decimali riportate.

Tutti i test sono superati. L'unica differenza formale è nel p-value del test di KSL: le tavole di Lilliefors usate da statsmodels hanno minimo 0.001, mentre JMP riporta < 0.01. La conclusione (normalità rifiutata) è la stessa.

6.8. Conclusioni

- In nessun dataset i residui della regressione lineare sono normali. Per questo la presenza di un trend è stata valutata con un test non parametrico (τ di Kendall) e la retta del trend è stata stimata con Theil-Sen, uno stimatore robusto.
- Un valore basso di R^2 non esclude la presenza di un trend: in EXP2, nelle metriche di I/O e nei VMres il trend è significativo anche con R^2 quasi nullo, perché è debole rispetto alla variabilità dei dati.
- Al contrario, un trend significativo per il test di Kendall può avere una pendenza di Theil-Sen non distinguibile da zero quando i dati sono a gradini o assumono pochi valori distinti (EXP2 nmail, os2 VmSize/VmData).
- Le metriche di memoria (VmSize, VmData, RSS, Allocated Heap) crescono nel tempo in tutti gli esperimenti, mentre le metriche di I/O non hanno un comportamento comune.
- Tra le tre macchine virtuali, VMres2 garantisce il tempo di saturazione dell'heap più lungo nel caso peggiore (almeno 46 anni), mentre VMres3 è la più critica (circa 11.7 anni).

7. BENCHMARK

7.1. Introduzione

Lo scopo del progetto è rilevare in modo scientificamente rigoroso e statisticamente significativo l'eventuale differenza di prestazioni tra due macchine con differenti caratteristiche hardware e profili energetici. Il carico di lavoro utilizzato per la valutazione è un benchmark di simulazione N-Body, un algoritmo a complessità computazionale $O(N^2)$ che modella l'evoluzione dinamica e le interazioni gravitazionali di un sistema di N corpi celesti.
I sistemi utilizati sotto esame é quello descritto nella tabella 2 e un secondo con simili caratteristiche ma versione desktop.

Tabella 30. Stime di Theil-Sen (os3).

Theil-Sen	VmSize	VmData	RSS	byte_letti_sec
Pendenza	0.0217	0.0217	0.00346	−1.891
Intercetta	744 374.599	675 970.599	45 386.471	141 291 080.771
IC 95%	[0.0213, 0.0220]	[0.0213, 0.0220]	[0.00339, 0.00352]	[−2.618, −1.168]

Tabella 31. Risultati della regressione OLS e del test di Mann-Kendall per `Allocated Heap` (VMres1–3).

Dataset	Retta OLS	R^2	RMSE	D (KSL)	τ	p (τ)	Trend
VMres1	$6.29650 \cdot 10^6 + 1.150\,x$	0.0337	1 045 360	0.0442	0.0700	< 0.0001	Sì
VMres2	$5.85082 \cdot 10^6 + 0.675\,x$	0.0248	661 297	0.1004	0.1378	< 0.0001	Sì
VMres3	$6.34573 \cdot 10^6 + 1.913\,x$	0.0521	1 285 300	0.0302	0.2031	< 0.0001	Sì

Tabella 32. Stime di Theil-Sen per `Allocated Heap` (VMres1–3).

Theil-Sen	VMres1	VMres2	VMres3
Pendenza (B/s)	0.691	0.671	2.903
Intercetta (B)	6 562 275.462	5 780 180.563	6 041 585.556
IC 95%	[0.564, 0.818]	[0.605, 0.737]	[2.715, 3.092]

Tabella 33. Tempo stimato al failure per `Allocated Heap` (VMres1–3).

Dataset	T_{failure} (s)	T_{failure} (anni)	Intervallo (anni)
VMres1	$1.544 \cdot 10^9$	49.0	[41.4, 60.0]
VMres2	$1.592 \cdot 10^9$	50.5	[46.0, 56.0]
VMres3	$3.678 \cdot 10^8$	11.7	[10.9, 12.5]

7.2. Analisi Descrittiva dei Dati Campionari

Per ciascun test sono stati analizzati $N = 31$ campioni per dispositivo. I tempi di esecuzione sono espressi in millisecondi (ms). Vedi tabella 34

7.3. Valutazione della Dimensione Campionaria Teorica (n)

Il calcolo della dimensione campionaria minima n necessaria per garantire la stima della media della popolazione entro un margine d'errore prefissato al livello di confidenza del 95% fornisce i seguenti valori:

Tabella 35. Numero minimo di campioni richiesti in funzione del margine di errore tollerato.

Errore tollerato	$N = 10\,000$ Laptop	$N = 10\,000$ Desktop	$N = 100\,000$ Laptop	$N = 100\,000$ Desktop
5%	$n = 2$	$n = 3$	$n = 1$	$n = 2$
8%	$n = 1$	$n = 1$	$n = 1$	$n = 1$
10%	$n = 1$	$n = 1$	$n = 1$	$n = 1$

Quindi la dimensione campionaria attuale ($n = 31$) è ampiamente sufficiente a garantire una stima accurata con errore inferiore al 5% per tutte le configurazioni.

7.4. Pipeline di Inferenza Statistica

Per la scelta del test d'ipotesi ottimale per ciascuna dimensione N, si valuta prima la normalità delle distribuzioni e successivamente l'omoschedasticità (uguaglianza delle varianze).

7.4.1. Test di Normalità (Shapiro-Wilk)

- Ipotesi Nulla (H_0): La distribuzione dei dati è normale ($\alpha = 0.05$).
- Configurazione $N = 10.000$:
  - Laptop: $W = 0.7988$, p-value $= 0.0000 \Rightarrow$ Non Normale
  - Desktop: $W = 0.5696$, p-value $= 0.0000 \Rightarrow$ Non Normale
- Configurazione $N = 100.000$:
  - Laptop: $W = 0.9424$, p-value $= 0.0963 \Rightarrow$ Normale
  - Desktop: $W = 0.7314$, p-value $= 0.0000 \Rightarrow$ Non Normale

7.4.2. Test di Omoschedasticità (Brown-Forsythe / Levene)

- Ipotesi Nulla (H_0): Le varianze dei due gruppi sono uguali.
- Per $N = 10.000$: $F = 0.1099$, p-value $= 0.7415 \Rightarrow$ Omoschedastico (Varianze statisticamente equivalenti).
- Per $N = 100.000$: $F = 4.2759$, p-value $= 0.0430 \Rightarrow$ Eteroschedastico (Varianze significativamente diverse, $p < 0.05$).

7.4.3. Confronto tra le prestazioni

Caso$N = 10.000$ Bodies Poiché i dati non seguono una distribuzione normale, il test di riferimento primario è il test non-parametrico di Wilcoxon, affiancato dal Welch t-test (valido in virtù del Teorema del Limite Centrale):

- Wilcoxon Rank-Sum Test: Statistica $Z = 4.2728$, p-value$= 1.93 \times 10^{-5}$
- Welch t-test (Unpooled): Statistica $t = 2.7119$, p-value$= 8.75 \times 10^{-3}$

Entrambi i test rifiutano l'ipotesi nulla di uguaglianza. Il sistema Desktop (media 205,45 ms) risulta statisticamente più veloce del sistema Laptop (media 210,84 ms).

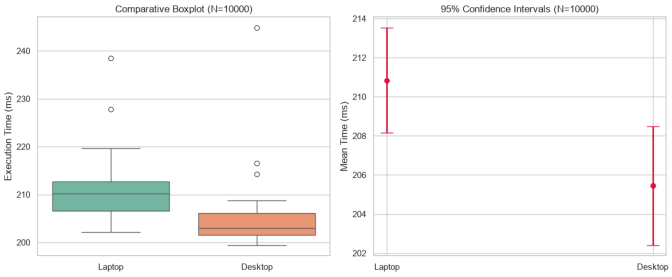


Figura 29. Caso $N = 10.000$ Bodies

Caso $N = 100.000$ Bodies In presenza di eteroschedasticità e non-normalità sul campione Desktop, si analizzano sia il test non-parametrico che il test di Welch per varianze ineguali:

- Welch t-test (Unpooled): Statistica $t = -7.8601$, p-value$= 1,85 \times 10^{-9}$
- Wilcoxon Rank-Sum Test: Statistica $Z = -5.2302$, p-value$= 1.69 \times 10^{-7}$

Vi è una differenza altamente significativa ($p < 0.0001$). In questa specifica prova, il sistema Laptop registra un tempo medio inferiore (23.407,46 ms) rispetto al Desktop (24.443,98 ms).

Tabella 34. Statistiche descrittive dei tempi di esecuzione per laptop e desktop al variare della dimensione del campione.

Parametro	$N = 10\,000$ **Laptop**	$N = 10\,000$ **Desktop**	$N = 100\,000$ **Laptop**	$N = 100\,000$ **Desktop**
Dimensione campione (n)	31	31	31	31
Media ($\bar{x}$)	210,84 ms	205,45 ms	23 407,46 ms	24 443,98 ms
Deviazione standard (s)	7,31 ms	8,28 ms	249,01 ms	690,71 ms
Errore standard (SE)	1,31 ms	1,49 ms	44,72 ms	124,05 ms
IC 95% inferiore	208,15 ms	202,42 ms	23 316,12 ms	24 190,63 ms
IC 95% superiore	213,52 ms	208,49 ms	23 498,80 ms	24 697,33 ms
Minimo	202,11 ms	199,45 ms	22 751,25 ms	22 078,98 ms
Q1 (25%)	206,63 ms	201,57 ms	23 258,73 ms	24 492,69 ms
Mediana	210,22 ms	203,01 ms	23 353,69 ms	24 722,68 ms
Q3 (75%)	212,74 ms	206,14 ms	23 520,89 ms	24 840,94 ms
Massimo	238,48 ms	244,79 ms	24 046,75 ms	25 054,94 ms

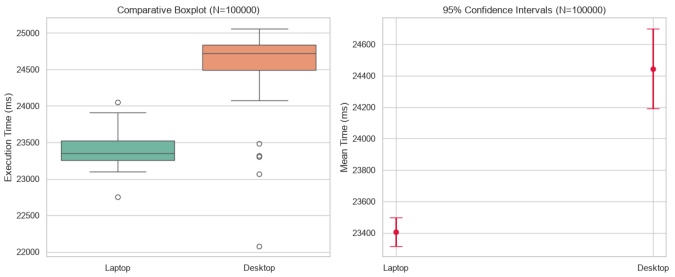


Figura 30. Caso $N = 100.000$ Bodies

8. FFDA

8.1. Introduzione

Il progetto in tale capitolo rigaurda Fast Failure Data Analysis (FFDA) per la misurazione, modellazione e caratterizzazione dell'affidabilità (Reliability & Dependability) di cluster di calcolo ad alte prestazioni (HPC), con focus sull'architettura Mercury e sui principi generali applicabili ad architetture come BlueGene.
Il sistema è implementato interamente in Python 3.13 mediante paradigma Object-Oriented Programming (OOP) e gestito tramite la toolchain Poetry.

8.2. Metodologia

Il flusso di lavoro riproduce il processo scientifico di analisi dei log di guasto di sistemi complessi:

- Ingestione e Filtraggio (Parsing & Filtering): Estrazione dei log di errore in formato syslog RedHat 9.0 e categorizzazione nei 6 domini principali del sistema (DEV, MEM, I-O, NET, PRO, OTH).
- Analisi di Coalescenza (CWIN Sensitivity): Raggruppamento temporale degli errori correlati mediante Coalescence Window (CWIN) e individuazione della dimensione ottima tramite regola del gomito (knee point).
- Calcolo dei Tempi al Guasto (TTF): Estrazione dei tempi di inter-arrivo (Δt) tra tuple di guasto consecutive.
- Reliability Empirica $R_{emp}(t)$: Stima non parametrica della probabilità di funzionamento senza guasti $P(TTF > t)$.
- Regressione Non-Lineare e Fit: Modellazione parametrica della Reliability mediante funzioni Esponenziali, Iper-esponenziali e di Weibull.
- Test di Validazione (Kolmogorov-Smirnov): Verifico statistico di aderenza della distribuzione teorica rispetto ai dati empirici.
- Stima del MTTF (Mean Time To Failure): Integrazione numerica della funzione di affidabilità ottimale.

8.3. Componenti

Data Models :

- `LogEntry`: Dataclass che modella una singola voce del log con timestamp, identificativo del nodo, categoria di errore e messaggio testuale.
- `CoalescedTuple`: Dataclass che unifica una sequenza di errori ravvicinati entro la finestra *CWIN*, calcolandone durata e dimensione.

Log Ingestion & Monitor :

- `LogFilterEngine`: Classifica e anonimizza il testo libero del syslog mappandolo nelle 6 categorie di errore:
  - DEV: Guasti ai dispositivi connessi (es. bus PCI).
  - MEM: Errore sui moduli di memoria RAM/ECC.
  - I-O: Errori di lettura/scrittura su dispositivi di storage.
  - NET: Perdita di connessione o link down.
  - PRO: Guasti ai processori/CPU.
  - OTH: Altri errori generici non classificati espressamente.
- `FileTailMonitor`: Classe concreta per il monitoraggio in tempo reale di file di log locali in style `tail -f`
- `SyslogServerMonitor`: Server socket UDP per la ricezione e il filtraggio remoto dei log inviati dai nodi del cluster.

Parser Engine

- `BaseLogParser`: Interfaccia astratta per l'ingestione e il filtraggio dei dataset per nodo o per categoria.
- `MercuryLogParser`: Parser specifico per i log dell'architettura a 3 livelli del cluster Mercury (Login, Compute, Storage).

Coalescence Engine

- `CoalescenceEngine`:
  - `tuple_logs(entries, cwin)`: Raggruppa eventi il cui tempo inter-arrivo è $\leq CWIN$
  - `calculate_inter_arrival_times(tuples)`: Estragga i vettori dei tempi di inter-arrivo (TTF) necessari per l'analisi di affidabilità.
  - `sensitivity_analysis(entries, cwin_range)`: Esegue il conteggio delle tuple al variare della finestra temporale ($CWIN \in [10s, 800s]$) per individuare il *knee point*

Reliability & Fitting Engine

- `ReliabilityAnalyzer`:
  - `compute_empirical_reliability(ttf_samples)`: ordina i campioni e genera la curva empirica $R_{emp}(t) = 1 - \frac{i}{N}$
  - Modelli Teorici di Fit:
    - Esponenziale: $f(t) = ae^{bt}$

    * Iper-esponenziale: $f(t) = ae^{bt} + (1-a)e^{dt}$ con $R(0) = 1.0$
    * Weibull: $f(t) = e^{-bt^a}$

- `fit_model()`: Esegue il fitting non-lineare sotto vincoli d'ambito (bounds), calcolando R^2, Errore Quadratico Medio (SSE) e la statistica del test di Kolmogorov-Smirnov (p-value)
- `calculate_mttf()`: Integra la curva di affidabilità $MTTF = \int_0^\infty R(t)dt$ mediante integrazione con regola del trapezio

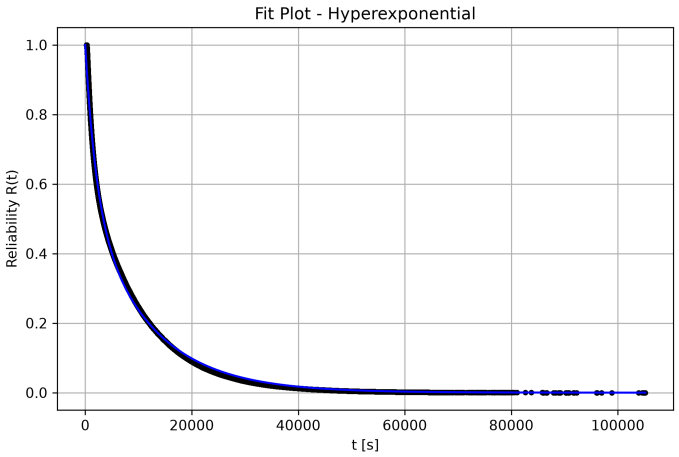


Figura 32. Riproduce l'andamento della curva empirica dei tempi al guasto sovrapposta al fit della funzione iper-esponenziale. La stima dell'MTTF a circa 2.04 ore

8.4. Risultati

In seguito all'elaborazione del dataset di test sul cluster Mercury, l'output generato a console riporta le seguenti metriche:

Code 1. Report dei Risultati

```
[*] Parsing log file: mercury_syslog.log...
[OK] Loaded 29899411 events.
[*] Execution CWIN Sensitivity Analysis ...
[OK] Plot CWIN saved in 'cwin_sensitivity.png'.
[OK] Generated 183242 tuples with CWIN=200s.

--- RESULTS HYPER-EXPONENTIAL REGRESSION ---
R^2: 0.99366
SSE: 96.77123
p-value (K-S Test): 0.0000
MTTF Stimato: 7339.31 secondi (2.04 ore)

[OK] Plot Reliability saved in 'reliability_fit.png'.
```

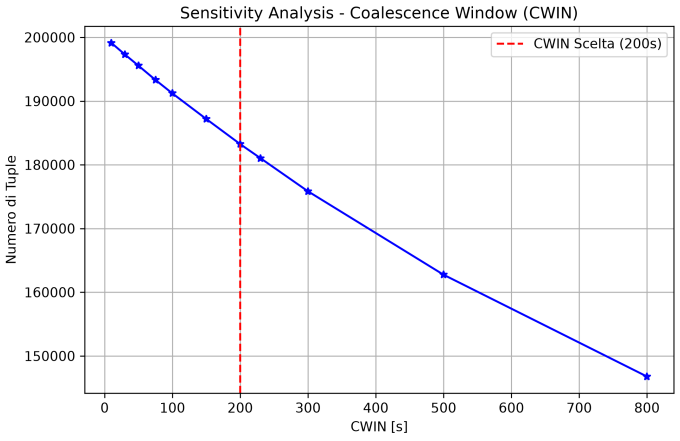


Figura 31. Mostra la variazione del numero di tuple in funzione della $CWIN$. Il gomito della curva individua la dimensione della finestra di coalescenza che minimizza il numero di collisioni senza frammentare eccessivamente i guasti correlati.

9. RBD & FAULT TREE

9.1. Esercizio 1

Il primo esercizio chiede di trovare la Reliability e il Mean Time To Failure (MTTF) di un sistema descritto dal diagramma a blocchi seguente. I componenti sono tutti identici e falliscono casualmente con il medesimo failure rate λ.

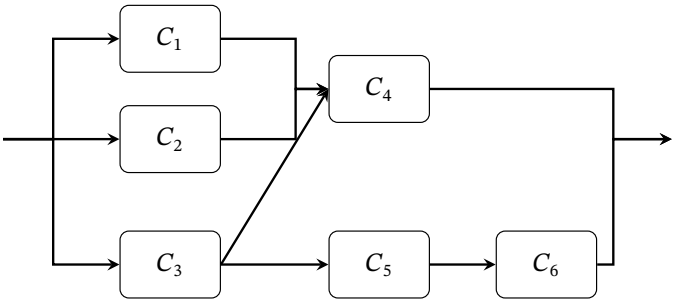


Figura 33. RBD iniziale del sistema dell'Esercizio 1.

Da un'analisi preliminare è possibile individuare il parallelo tra i componenti 1 e 2 e la serie tra 5 e 6:

$$\begin{aligned} R_{1,2} &= 1 - \prod_{i=1}^{2}(1-R_i) \\ &= 1-(1-R_1)(1-R_2) = 1-(1-R)^2 \end{aligned} \tag{18}$$

$$R_{5,6} = \prod_{i=5}^{6} R_i = R_5 R_6 = R^2 \tag{19}$$

A questo punto si applica il metodo del *Conditioning* scegliendo il componente 3:

- **Caso 1 (C_3 funziona / corto circuito):** $C_{5,6}$ e C_4 risultano in parallelo:

$$\begin{aligned} R_{4,5,6} &= 1-(1-R_4)(1-R_5R_6) \\ &= 1-(1-R)(1-R^2) \\ &= R+R^2-R^3 \end{aligned} \tag{20}$$

- **Caso 2 (C_3 non funziona / circuito aperto):** Rimane la serie tra $C_{1,2}$ e C_4:

$$\begin{aligned} R_{1,2,4} &= R_{1,2}R_4 \\ &= (1-(1-R_1)(1-R_2))\,R_4 \\ &= (2R-R^2)R = 2R^2-R^3 \end{aligned} \tag{21}$$

Applicando il Teorema della Probabilità Totale:

$$\begin{aligned} R_{sys} &= R_3 P(\text{Funziona} \mid C_3 \text{ funziona}) \\ &\quad + (1-R_3)P(\text{Funziona} \mid C_3 \text{ non funziona}) \\ &= R_3 R_{4,5,6} + (1-R_3)R_{1,2,4} \\ &= R(1-(1-R)(1-R^2)) \\ &\quad + (1-R)(1-(1-R)^2)R \\ &= -2R^3 + 3R^2 \end{aligned} \tag{22}$$

Assumendo una distribuzione dei guasti esponenziale ($R(t) = e^{-\lambda t}$):

$$R_{sys}(t) = -2e^{-3\lambda t} + 3e^{-2\lambda t} \tag{23}$$

Integrando la Reliability si ricava il *Mean Time To Failure* (MTTF):

$$\begin{aligned} \text{MTTF} &= \int_0^\infty R_{sys}(t)\,dt \\ &= \int_0^\infty \left(-2e^{-3\lambda t} + 3e^{-2\lambda t}\right) dt \\ &= \left[\frac{2}{3\lambda}e^{-3\lambda t} - \frac{3}{2\lambda}e^{-2\lambda t}\right]_0^\infty \\ &= \frac{3}{2\lambda} - \frac{2}{3\lambda} = \frac{5}{6\lambda} \end{aligned} \tag{24}$$

Infine, applicando il teorema dell'Upper-Bound per la validazione:

$$\begin{aligned} R_{sys} &\le 1 - \prod_i (1 - R_{path,i}) \\ &= 1 - (1-R^2)^3(1-R^3) \\ &= 3R^2 + R^3 - 3R^4 - 3R^5 \\ &\quad + R^6 + 3R^7 - R^9 \end{aligned} \tag{25}$$

9.2. Esercizio 2

Il secondo esercizio chiede di derivare le espressioni analitiche della Reliability per due distinte architetture ridondanti:

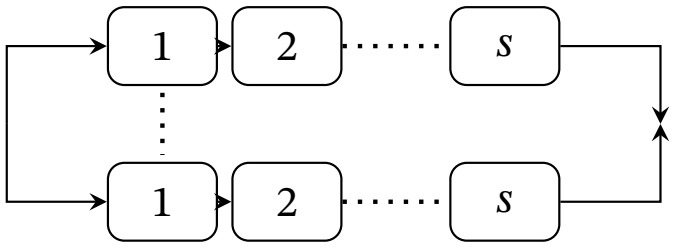


Figura 34. Sistema 1: m catene parallele di s componenti in serie.

Il primo sistema origina da m catene parallele di s componenti:

$$\begin{aligned} R_{chain}(t) &= \prod_{i=1}^{s} R_i = R^s \\ \Rightarrow\ R_{sys1}(t) &= 1 - \prod_{i=1}^{m}(1 - R_{chain,i}) \\ &= 1 - (1-R^s)^m \end{aligned} \tag{26}$$

Il secondo sistema origina dalla serie di s blocchi, ciascuno formato da m componenti in parallelo:

$$\begin{aligned} R_{block}(t) &= 1 - \prod_{i=1}^{m}(1-R) = 1 - (1-R)^m \\ \Rightarrow\ R_{sys2}(t) &= \prod_{i=1}^{s} R_{block,i} = \left(1-(1-R)^m\right)^s \end{aligned} \tag{27}$$

Ponendo $m = 2$ ed $s = 4$:

$$R_{sys1}(t) = 1 - (1-R^4)^2 = -R^8 + 2R^4 \tag{28}$$

$$\begin{aligned} R_{sys2}(t) &= \left(1-(1-R)^2\right)^4 \\ &= R^8 - 8R^7 + 24R^6 - 32R^5 + 16R^4 \end{aligned} \tag{29}$$

Calcolando le Reliability all'istante $t = \text{MTTF}$ (in cui $R = e^{-1}$):

$$R_{sys1}(\text{MTTF}) = 1 - (1-e^{-4})^2 \approx 0.036 = 3.6\% \tag{30}$$

$$R_{sys2}(\text{MTTF}) = \left(1-(1-e^{-1})^2\right)^4 \approx 0.13 = 13\% \tag{31}$$

Per allineare la Reliability del primo sistema a quella del secondo (0.13), si calcola il numero m di catene parallele necessarie:

$$\begin{aligned} 1 - (1-e^{-4})^m = 0.13 \ &\Rightarrow\ (1-e^{-4})^m = 0.87 \\ &\Rightarrow\ m \approx 8 \end{aligned} \tag{32}$$

9.3. Esercizio 3

In questo esercizio si analizza la Reliability di una rete *Skip-Ring* a 8 nodi. L'operatività è garantita se almeno 4 nodi operano correttamente e purché non si guastino mai due nodi adiacenti consecutivi.

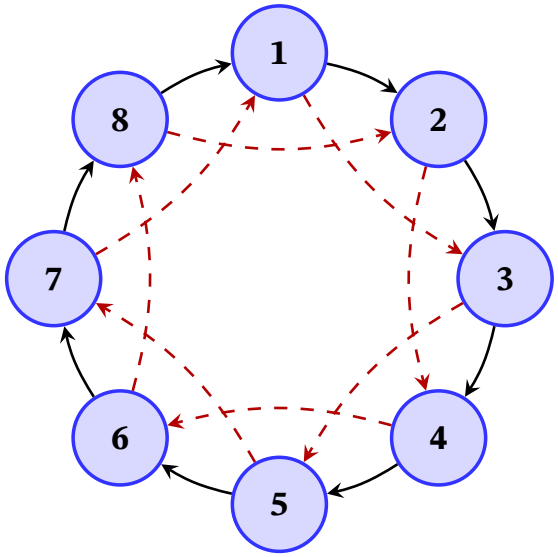


Figura 35. Topologia della rete Skip-Ring a 8 nodi.

Si valuta la probabilità di guasto al variare del numero di nodi non funzionanti:

Tabella 36. Valutazione delle combinazioni di guasto della rete Skip-Ring.

Nodi guasti	Comb. tot.	Comb. non funz.	$P(n$ nodi fall.)	$P(\text{guasto} \mid n)$
0	1	0	R_m^8	0
1	8	0	$8R_m^7(1-R_m)$	0
2	28	8	$28R_m^6(1-R_m)^2$	8/28
3	56	40	$56R_m^5(1-R_m)^3$	40/56
4	70	68	$70R_m^4(1-R_m)^4$	68/70
5	56	Tutte	$56R_m^3(1-R_m)^5$	1
6	28	Tutte	$28R_m^2(1-R_m)^6$	1
7	8	Tutte	$8R_m(1-R_m)^7$	1
8	1	Tutte	$(1-R_m)^8$	1

Applicando il teorema della probabilità totale, si ricava l'Unreliability $F(t)$:

$$\begin{aligned} F(t) &= 8R_m^6(1-R_m)^2 + 40R_m^5(1-R_m)^3 \\ &\quad + 68R_m^4(1-R_m)^4 + 56R_m^3(1-R_m)^5 \\ &\quad + 28R_m^2(1-R_m)^6 + 8R_m(1-R_m)^7 \\ &\quad + (1-R_m)^8 \end{aligned} \tag{33}$$

La Reliability risulta $R(t) = 1 - F(t)$. Assumendo failure rate $\lambda = 0.005\,\text{h}^{-1}$, dopo $t = 48$ h si ha $R_m(48\text{h}) = e^{-0.24} \approx 0.7866$, da cui:

$$R(48\text{h}) \approx 0.7289 \tag{34}$$

9.4. Esercizio 4

Si analizza e si confronta la Reliability di 4 coppie di sistemi, assumendo distribuzioni dei guasti esponenziali con i seguenti MTTF: $\text{MTTF}_A = 500$ h, $\text{MTTF}_B = 9000$ h, $\text{MTTF}_C = 10000$ h.

Prima coppia :

$$R_{sys1}(t) = 1 - (1 - R_A R_B)(1 - R_A R_C) \tag{35}$$

$$R_{sys2}(t) = (1 - (1 - R_B)(1 - R_C))\, R_A \tag{36}$$

Dall'analisi delle curve si deduce che $R_{sys1}(t) > R_{sys2}(t)$.

Seconda coppia :

$$R_{sys1}(t) = (1 - (1 - R_B)(1 - R_A))\, R_A \tag{37}$$

$$R_{sys2}(t) = R_A \tag{38}$$

In questo caso si deduce che $R_{sys2}(t) > R_{sys1}(t)$.

Terza coppia :

$$R_{sys1}(t) = (1 - (1 - R_B)(1 - R_A))\,(R_A R_B) \tag{39}$$

$$R_{sys2}(t) = R_A R_B \tag{40}$$

Risulta $R_{sys2}(t) > R_{sys1}(t)$.

Quarta coppia :

$$R_{sys1}(t) = 1 - (1 - R_A R_B)(1 - R_A) \tag{41}$$

$$R_{sys2}(t) = R_A \tag{42}$$

In questo caso si deduce che $R_{sys1}(t) > R_{sys2}(t)$.

9.5. Esercizio 5

Viene assegnato l'apparato avionico di pilotaggio di un elicottero composto da Processing Unit (P), Remote Terminal (T), AHRS (AH), Doppler (DP), INS (INS) e Bus (BA, BB).

Punto A : Costruzione del Reliability Block Diagram (RBD).

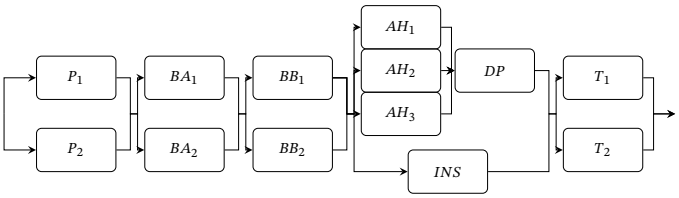


Figura 36. Reliability Block Diagram dell'apparato avionico.

Punto B : Costruzione dell'Albero dei Guasti (Fault Tree) e determinazione dei Minimal Cut Sets.

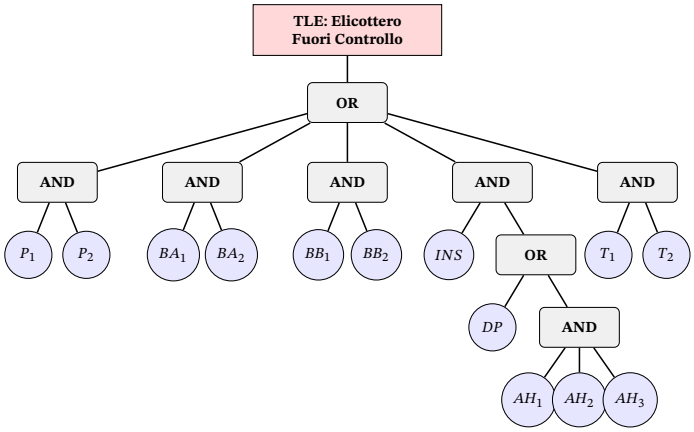


Figura 37. Fault Tree per il sistema avionico dell'elicottero.

La funzione booleana di guasto del sistema è:

$$\begin{aligned} T(x) = {} & (P_1 \wedge P_2) \vee (BA_1 \wedge BA_2) \vee (BB_1 \wedge BB_2) \\ & \vee (INS \wedge (DP \vee (AH_1 \wedge AH_2 \wedge AH_3))) \\ & \vee (T_1 \wedge T_2) \end{aligned} \tag{43}$$

I *Minimal Cut Sets* (MCS) sono pertanto i seguenti:

- $(P_1 \wedge P_2)$
- $(BA_1 \wedge BA_2)$
- $(BB_1 \wedge BB_2)$
- $(INS \wedge DP)$
- $(INS \wedge AH_1 \wedge AH_2 \wedge AH_3)$
- $(T_1 \wedge T_2)$

Punto C : Calcolo dell'affidabilità dopo 1 ora di volo. Dati gli MTTF (Processing Unit: 10000 h, Remote Terminal: 4500 h, AHRS: 2000 h, INS: 2000 h, Doppler: 500 h, Bus: 60000 h):

$$\begin{aligned} R(t) = {} & \left(1 - (1 - R_P)^2\right) \left(1 - (1 - R_B)^2\right)^2 \\ & \cdot \left(1 - (1 - R_{INS}) \left(1 - R_{DP}(1 - (1 - R_{AHRS})^3)\right)\right) \\ & \cdot \left(1 - (1 - R_T)^2\right) \end{aligned} \tag{44}$$

Valutando a $t = 1$ h:

$$R(1\text{h}) \approx 0.99999 \quad (5 \text{ nines}) \tag{45}$$

Punto D : Incorporazione del fattore di *coverage* c per la Processing Unit.

$$\begin{aligned} R_c(1\text{h}) = {} & (1 - (1 - R_P)(1 - R_P \cdot c)) \\ & \cdot \left(1 - (1 - R_B)^2\right)^2 \cdot R_{\text{resto}}(1\text{h}) = 0.9999 \end{aligned} \tag{46}$$

Risolvendo la relazione rispetto a c:

$$c = 0.910079 \quad (91.01\%) \tag{47}$$

RIFERIMENTI BIBLIOGRAFICI

[1] T. Zhang, R. Ramakrishnan e M. Livny, «BIRCH: an efficient data clustering method for very large databases,» *SIGMOD Rec.*, vol. 25, n. 2, pp. 103–114, giu. 1996, ISSN: 0163-5808, doi: 10.1145/235968.233324. indirizzo: https://doi.org/10.1145/235968.233324